

Title Page

Everything ATHO: Full Architecture, Consensus, Network, Codebase, Security, Testing, and Implementation Analysis

Project name: Atho
Asset / ticker: ATHO
Document type: Full Technical Architecture and Consensus Analysis
Audience: Engineers, auditors, protocol developers, infrastructure operators, node operators, miners, and wallet developers
Version / date: 2026-05-18
Author / organization: Open technical draft; final publication owner to be assigned

Abstract

Atho is a payment-focused Layer 1 blockchain implemented primarily in Rust and organized around a UTXO accounting model, Proof-of-Work block production, deterministic node-side validation, Falcon-512 post-quantum signatures, and SHA3-family hashing. The codebase aims to stay close to the operational virtues of Bitcoin-like systems while replacing classical signature assumptions with a post-quantum ownership model and retaining a relatively small base-layer attack surface. In the current repository, Atho includes a full node, mempool, miner, wallet, API/RPC layer, P2P transport, LMDB-backed storage, a desktop client, and a growing adversarial and regression test suite. This paper explains the implemented architecture, the consensus pipeline, how transactions and blocks are formed and validated, how chainstate mutates, how reorgs are handled, how network separation is enforced, and how storage, mining, and wallet construction interact. It also incorporates the project's internal consensus and edge-case audits to separate implemented behavior from planned behavior and to state clearly where mainnet confidence is strong and where it still needs more proof. The central conclusion is that

Atho's consensus core is materially stronger than an experimental toy chain and already exhibits several production-minded design choices, but the surrounding proof net, differential testing, fuzz execution cadence, and some hardening surfaces still warrant additional work before a mainnet launch should be called routine.

Table of Contents

- Title Page
- Abstract
- 1. Introduction
- 2. High-Level Architecture
- 3. Codebase Map
- 4. Consensus Overview
- 5. Consensus-Critical Rules List
- 6. Block Structure
- 7. Block Validation Pipeline
- 8. Transaction Structure
- 9. Transaction Lifecycle
- 10. Transaction Validation
- 11. UTXO Model
- 12. Address and HPK System
- 13. Falcon-512 Signature System
- 14. Hashing System
- 15. Serialization and Canonical Encoding
- 16. Merkle Roots and Transaction Ordering
- 17. Proof-of-Work
- 18. Difficulty Adjustment
- 19. Monetary Policy and Emissions
- 20. Coinbase Transaction

- 21.Fees, Dust, Weight, and Vsize
- 22.Mempool
- 23.Miner
- 24.Block Propagation and Network Layer
- 25.Sync Strategy
- 26.Chainstate and Storage
- 27.Reorgs and Fork Choice
- 28.API/RPC Layer
- 29.Wallet Architecture
- 30.Mainnet, Testnet, and Regtest Separation
- 31.Security Model
- 32.Consensus Vulnerability Analysis
- 33.Edge-Case Testing Plan
- 34.Adversarial Transaction Test Plan
- 35.Adversarial Block Test Plan
- 36.Fuzz Testing Plan
- 37.Property-Based Testing Plan
- 38.Differential Testing Plan
- 39.Storage Crash Testing Plan
- 40.Legacy and Bypass Audit
- 41.Performance and Scalability
- 42.Code Examples
- 43.Production Readiness Grading
- 44.Mainnet Blocking Conditions
- 45.Mainnet Launch Checklist
- 46.Operational Deployment
- 47.Governance and Upgrade Safety
- 48.Current Risks and Open Issues
- 49.Glossary

50.Conclusion

- References
- Appendix A. Full Consensus Rule List
- Appendix B. Full Test Matrix
- Appendix C. Full Code Module Map
- Appendix D. Full Vulnerability Table
- Appendix E. Open Questions
- Appendix F. Future Improvements

List of Tables

1. Table 1. System component summary.
2. Table 2. Major codebase modules.
3. Table 3. Consensus failure classes.
4. Table 4. Consensus rule categories.
5. Table 5. Address and ownership fields.
6. Table 6. Hashing roles.
7. Table 7. Database responsibilities.
8. Table 8. Network message classes.
9. Table 9. API/RPC trust boundary summary.
10. Table 10. Production readiness grading.
11. Table 11. Current open risks.
12. Table 12. Mainnet launch gate summary.

List of Figures

1. Figure 1. End-to-end Atho transaction and block lifecycle.
2. Figure 2. High-level Atho subsystem diagram.

3. Figure 3. Canonical block validation pipeline.
4. Figure 4. UTXO state transition model.
5. Figure 5. Reorg disconnect/reconnect flow.

1. Introduction

Atho is best understood as a post-quantum UTXO blockchain that deliberately keeps the consensus surface narrower than a general-purpose smart-contract platform. The codebase favors full-node validation, explicit serialization rules, PoW-based chain ordering, and storage-backed chainstate over account-style execution complexity or VM-level programmability. That choice has two immediate consequences. First, the base protocol is easier to reason about because a spend either consumes an identified unspent output or it does not; there is no general mutable account state machine in the consensus core. Second, the engineering burden shifts toward rigorous validation, deterministic data formats, reorg safety, and wallet correctness rather than contract execution metering or VM semantics.

The project's stated and implemented emphasis is payments. In practice that means low-friction transaction creation, deterministic spend authorization, modest fee and dust defenses, and a node that can validate every block itself. Atho does not currently present itself as a zero-knowledge privacy chain, a Turing-complete contract chain, or a delegated validator network. The base-layer trust model is still classical Nakamoto-style chain growth plus node-side validation, but the spend authorization primitive is Falcon-512 rather than an elliptic-curve signature scheme. That gives Atho a distinct architectural identity: the system is Bitcoin-like in operational shape, but explicitly post-quantum in the ownership layer.

That distinctiveness matters because post-quantum blockchains can drift into one of two traps. One trap is to keep the old system architecture but swap primitives without re-auditing the consequences. The other is to add so much novelty at once that consensus review becomes intractable. Atho is more conservative than that. It uses SHA3-family hashing across multiple digest roles, Falcon signatures for authorization, a UTXO state machine for spend tracking, and a

miner-driven PoW chain for ordering. The code repeatedly favors canonical encodings, typed validation, and network separation rules over flexible or ambiguous object formats.

Compared with Bitcoin-like systems, Atho stays close in the areas that support reliability: block assembly, UTXO application, staged validation, proof-of-work chain selection, coinbase rewards, and a strong distinction between policy and consensus. Compared with account-based VM chains, Atho avoids global account mutation, contract gas semantics, and stateful execution at the transaction layer. Compared with more experimental post-quantum designs, Atho is not trying to solve everything at once. That restraint is a strength. It narrows the set of ways consensus can fail.

What Atho tries to solve is straightforward: offer a payment-centric blockchain whose ownership model is oriented toward post-quantum cryptography, whose node implementation is auditable, and whose operator surfaces remain practical. What Atho does not currently solve includes expressive smart contracts, shielded on-chain privacy, trustless complex script semantics, or magically removing the operational realities of PoW, custody, and endpoint compromise. The right way to evaluate Atho, therefore, is not as a platform that aims to do everything. It is as a deliberately narrow chain where consensus determinism, wallet correctness, PoW validation, storage safety, and replay-resistant network separation have to carry most of the system's weight.

2. High-Level Architecture

At a system level, Atho is composed of a wallet/key-management surface, a transaction builder and signer, a node-side mempool and chainstate, a miner block-template path, a block validator, an LMDB-backed persistence layer, and RPC/API plus P2P ingress surfaces. The node is the authority for consensus acceptance. Wallets may construct valid transactions, miners may assemble candidate blocks, and peers may relay objects, but none of those actors can mutate the canonical state until the node's validation path accepts the object.

Figure 1 shows the intended lifecycle.


```
| UTXOs, blocks, DB |
+-----+
```

Table 1 summarizes the major components.

Component	Purpose	Consensus-Critical?	Main Risks	Code Area
atho-core	Protocol types, hashes, serialization, consensus constants	Yes	Hash drift, serialization drift, constant drift	crates/atho-core
atho-storage	UTXO state, block acceptance, reorgs, LMDB persistence	Yes	Dirty writes, reorg corruption, replay mismatch	crates/atho-storage
atho-node	Mempool, miner templates, runtime, API entrypoints	Yes	Policy/consensus drift, startup failure handling	crates/atho-node
atho-p2p	Peer transport, block and tx relay, compact block handling	Yes	Wrong-network parsing, compact reconstruction bugs	crates/atho-p2p
atho-wallet	Key derivation,	Wallet-level	but Wrong wrong sighash, change,	crates/atho-

Component	Purpose	Consensus-Critical?	Main Risks	Code Area
	address generation, spend building, signing	security-sensitive	wrong-network tx creation	wallet
atho-rpc	RPC command model transport encoding	Non-consensus but security-sensitive	Validation bypass, malformed request handling	crates/atho-rpc
atho-qt	Desktop interface and local client controls	Non-consensus	User confusion, unsafe defaults, display drift	crates/atho-qt
Fuzz tooling	Parser, audit validator, adversarial harnesses	No direct consensus, but critical for proof	Coverage gaps	fuzz/, audit docs

The engineering point is simple: Atho’s architecture only works if the consensus-critical path stays short, deterministic, and isolated from convenience surfaces. Every external ingress, whether from a wallet, API, miner, or peer, must be forced through the same canonical parsing and validation machinery before chainstate moves.

3. Codebase Map

The repository is organized into protocol, storage, node, wallet, networking, UI, and supporting crates. The most important boundary is between object definition and object acceptance. `atho-core` defines what blocks, headers, transactions, addresses, hashes, and rule constants look like. `atho-storage` defines how those objects are validated against live state. `atho-node` wraps that validation in a mempool, miner, runtime, sync engine, and service façade. `atho-p2p` handles external message movement, while `atho-wallet` constructs spends that must still satisfy the validator’s rules.

Table 2 highlights the major modules that matter most to consensus and operations.

File / Module	Responsibility	Consensus Role	Security Risk	Test Coverage	Notes
<code>crates/atho-core/src/block.rs</code>	Block/header type definitions, merkle and witness roots, canonical bytes	Critical	Header drift can split consensus	Good but still benefits from differential coverage	Commits block identity
<code>crates/atho-core/src/transaction.rs</code>	Transaction bytes, txid/wtxid, witness commitment logic	Critical	Serialization drift or omitted committed fields can split consensus	Good	Current code binds tx-PoW through witness commitment
<code>crates/atho-core/src/</code>	Network prefixes, IDs, ports, magic bytes	Critical	Cross-network contamination	Good	Includes Mainnet/Testnet/R egnet/Prunetest

File / Module	Responsibility	Consensus Role	Security Risk	Test Coverage	Notes
network.rs					
crates/atho-core/src/consensus/pow.rs	Target rules, chainwork, checks	PoW Critical	Wrong target more math or differential ordering bugs	Good, but work is useful	Validators must recompute hashes
crates/atho-core/src/consensus/subsidy.rs	Monetary schedule	Critical	Inflation reward drift	/ Good	Must stay deterministic forever
crates/atho-core/src/consensus/signatures.rs	Domain-separated signature prehashes	Critical	Signing/ verifying mismatch	Good	Anchors Falcon message rules
crates/atho-storage/src/validation.rs	Transaction and block validation against live state	Critical	Invalid accept / valid reject	Strong targeted coverage	Consensus heart of the node
crates/atho-storage/src/utxo.rs	UTXO apply/disconnect logic, maturity checks	Critical	Double spend / rollback corruption	Strong and improving	Atomicity here is mandatory
crates/atho-storage/src/	Canonical chain tracking, reorgs,	Critical	Reorg corruption,	Improving with	Stores the active

File / Module	Responsibility	Consensus Role	Security Risk	Test Coverage	Notes
chainstate.rs	snapshot import/export		dirty restart mismatch property tests		chain model
crates/atho-storage/src/db.rs	LMDB and block-file persistence, commit journal, recovery checks	Critical	Partial trust, write restart corruption	Better after failpoints	Operationally sensitive
crates/atho-node/src/mempool.rs	Relay policy, double-spend filtering, mining candidate inputs	High	Policy/consensus drift	Good	Must not substitute for block validation
crates/atho-node/src/mining.rs	Candidate block assembly	Critical	Miner/validator disagreement	Moderate	Candidate must pass the normal validator
crates/atho-node/src/node.rs	In-process node state machine and block connect path	Critical	Startup wrappers, snapshot bootstrap auth	Moderate to good	Key runtime façade
crates/atho-node/src/service.rs	RPC/API-facing node service	High	Validation bypass ingress	Good at depth	test Policy layer around consensus engine
crates/atho-node/src/	Process startup, listeners, cookie/RPC auth	Security-sensitive	Panic on load, auth drift	Moderate	Non-consensus but production-critical

File / Module	Responsibility	Consensus Role	Security Risk	Test Coverage	Notes
runtime.rs					
crates/atho-node/src/sync.rs	Sync coordination and chain competition handling	Critical-adjacent	Fork handling, stale state reuse	Good	Important for network correctness
crates/atho-p2p/src/protocol.rs	Wire formats, compact blocks, message validation	Critical-adjacent	Parsing / reconstruction ambiguity	Improving	Must never create consensus divergence
crates/atho-wallet/src/wallet.rs	Spend builder, signer grouping, tx-PoW solving	Wallet-level but wrong security-sensitive	Wrong fee, sighash, invalid witness refs	Good local tests; differential work still thin	Produces transactions that must survive node validation
crates/atho-qt/src/app	Desktop UX and node controls	Non-consensus	Operational unsafe defaults	Growing	Operator-facing surface

The codebase is already large enough that documentation must distinguish between consensus, policy, wallet, storage, and UI behavior. A reader should never infer that a mempool rule is automatically a block-validity rule, or that a wallet-side choice is automatically enforced by full nodes. That separation is one of the most important themes in the repository.

4. Consensus Overview

Consensus in Atho is the set of rules that lets two honest nodes given the same prior state and the same candidate block reach the same accept/reject result and the same post-state. That definition sounds obvious, but it is the thread that ties the entire implementation together. Block structure, transaction bytes, witness parsing, hashing, difficulty, UTXO lookups, fee arithmetic, coinbase checks, storage commits, and reorg rollback are all consensus-relevant if a mismatch can make nodes disagree.

Table 3 captures the core failure classes.

Consensus Failure	Cause	Impact	Required Defense
Invalid block accepted	Missing or inconsistent validation	Inflation, double spend, chain split	Full deterministic validation
Valid block rejected	Non-determinism, stale rule path, serialization drift	Chain split	Single canonical rules and extensive tests
Double spend	UTXO spend authorization failure	Monetary loss	Exact UTXO checks and duplicate-input rejection
Coinbase overclaim	Wrong fee or subsidy math	Inflation	Exact subsidy + valid fees rule
Signature bypass	Wrong message binding or skipped verification	Theft	Strict Falcon verification path
Reorg corruption	Disconnect/connect mismatch	Wrong balances, wrong tip	Undo discipline and replay checks
Storage	Bad restart checks	Silent divergence	Startup integrity

Consensus Failure	Cause	Impact	Required Defense
corruption trusted			verification and reindex
Cross-network replay	Weak network separation	Wrong-network object acceptance	Consensus IDs, prefixes, magic bytes, genesis anchors

Atho's implementation already internalizes a number of the right principles. Validators recompute block hashes instead of trusting miners. Transaction validation is split between context-free structure checks and context-dependent UTXO authorization. Coinbase rules are enforced in block validation rather than delegated to miners. The chainstate layer treats persistence as a consequence of successful validation rather than the source of validity. Reorgs explicitly disconnect and reconnect through validated state transitions.

The remaining hard work is mostly about proof and exhaustiveness rather than inventing a new consensus shape. The system needs enough fuzzing, differential testing, crash-window coverage, and startup integrity checking to justify confidence that the implemented rules are the only rules nodes will actually follow.

5. Consensus-Critical Rules List

Atho's rules divide naturally into object rules, state rules, chain rules, and environment-separation rules. Block headers must serialize canonically and hash deterministically. Transactions must use the accepted version and canonical bytes, include valid witness data for normal spends, and satisfy transaction PoW, fee, size, and dust-related constraints at the right layer. UTXOs must exist, be unspent, belong to the signer, and be mature when coinbase-derived. Coinbase outputs must not exceed the exact allowed subsidy plus valid fees. Chain selection must follow cumulative work rather than raw height. Mainnet and non-mainnet data

must remain mutually invalid.

Table 4 groups the rule families.

Rule	Area	Consensus-Critical?	Failure Impact	Required Test
Canonical block bytes	Serialization	Yes	Hash mismatch / Roundtrip split	+ trailing-byte rejection
Canonical tx bytes	Serialization	Yes	Txid drift / split	Roundtrip + malformed decode rejection
Supported version at height	Ruleset activation	Yes	Valid reject / invalid accept	Boundary-version tests
Network match	ID Network separation	Yes	Replay / wrong-chain accept	Wrong-network block tests
Duplicate input rejection	Transaction block validity	/ Yes	Double spend	Same-tx and same-block duplicate tests
Coinbase first and unique	Block validity	Yes	Inflation or invalid blocks	No-coinbase / two-coinbase tests
Coinbase maturity	UTXO spend authorization	Yes	Economic safety failure	One-block-too-early tests
Fee = inputs - outputs	Monetary accounting	Yes	Inflation or burn drift	Exact-fee tests
Coinbase subsidy + valid	$\leq$ Monetary	Yes	Inflation	Overclaim-by-one tests

Rule	Area	Consensus-Critical?	Failure Impact	Required Test
fees	policy			
PoW check	target Header validity	Yes	Invalid-block acceptance	Above/equal/below target tests
Reorg exactness	rollback Chainstate	Yes	Silent balance corruption	Replay-equals-live tests
Runtime network constant separation	Environment	Yes	Cross-network contamination	Mainnet/testnet/regnet mismatch tests

A more exhaustive rule inventory appears in Appendix A, while Appendices B through D include deeper test and vulnerability material from the internal audits.

6. Block Structure

Atho blocks contain a header and an ordered transaction list. The header includes the block version, network identifier, height, previous block hash, merkle root, witness root, founders-hash commitments, timestamp, difficulty target, and nonce. The body contains the coinbase transaction followed by zero or more normal transactions. The code implements these structures in `crates/atho-core/src/block.rs`, where canonical bytes and block-hash derivation are defined in one place.

A representative real-code shape is closer to the following than to a minimal Bitcoin clone:

```
pub struct BlockHeader {
```

```
pub version: u16,  
pub network_id: Network,  
pub height: u64,  
pub previous_block_hash: [u8; 48],  
pub merkle_root: [u8; 48],  
pub witness_root: [u8; 48],  
pub founders_hash_sha3_384: [u8; 48],  
pub founders_hash_sha3_512: [u8; 64],  
pub timestamp: u64,  
pub difficulty_target_or_bits: [u8; 48],  
pub nonce: u64,  
}
```

The consensus-critical point is that headers commit to the exact information validators need to judge chain order and body commitment. The previous block hash binds ancestry, the merkle root binds the transaction ordering, and the witness root binds witness-related transaction identity material. The header is serialized canonically, and the validator recomputes the header hash rather than trusting a miner-reported value.

Block acceptance is intentionally staged. Structural checks and commitment checks run before chainstate mutation. Contextual checks then verify the parent hash, expected height, expected target, timestamp bounds, and PoW target satisfaction. Only after the block body passes transaction revalidation and the coinbase amount check does the chainstate layer persist the new state.

7. Block Validation Pipeline

The block validator is the core of Atho's safety model. A fully validated block is not just well-formed bytes; it is a coherent state transition from one exact chain tip to the next.

Figure 3. Canonical block validation pipeline.

Receive block

- > decode canonical bytes
- > verify network and version
- > verify expected parent, height, and target
- > verify timestamp bounds
- > verify proof of work
- > verify merkle root and witness root
- > verify one coinbase and coinbase-first ordering
- > verify transaction IDs and input uniqueness
- > revalidate every non-coinbase transaction against a staged UTXO overlay
- > sum valid fees
- > verify coinbase reward $\leq$ subsidy + valid fees
- > commit the state transition atomically

The validator in `atho-storage/src/validation.rs` and the chainstate apply path in `atho-storage/src/chainstate.rs` together enforce this pipeline. The important design choice is that transaction validity inside a block is not inherited from mempool admission. Every transaction is checked again against the active chain tip and a staged UTXO overlay. That is the only safe design because the mempool can be stale, miners can reorder transactions, and peers can submit blocks containing transactions the local node never relayed.

```
validate_block(block, chainstate):
```

```
    reject malformed bytes
```

```
    reject wrong network, wrong height, wrong parent, wrong target, bad pow
```

```
    reject merkle or witness commitment mismatch
```

```
    reject zero or multiple coinbases
```

```
    reject duplicate txids or duplicate inputs
```

```
    stage UTXO overlay from live state
```

```
    for each normal tx:
```

```
        validate structure, fee floor, witness, tx-pow, and contextual spend  
rights
```

```
        apply spends and outputs to overlay
```

```
        accumulate fees
```

```
    allowed_reward = subsidy(height) + total_fees
```

```
    reject if coinbase exceeds allowed_reward
```

```
atomically persist block + UTXO snapshot + tip update
```

This staged model is non-negotiable. If validation mutated the live chainstate incrementally and then failed partway through, the node could silently diverge from honest peers or present the wrong balances to operators.

8. Transaction Structure

Atho transactions are UTXO spends with explicit inputs, outputs, a lock time, a serialized witness payload, and transaction PoW fields. Inputs reference a previous transaction ID and output index plus a canonical unlocking script that, in current Atho, is effectively the 32-byte ownership lock for the UTXO being spent. Outputs carry an amount and a locking script. The witness carries Falcon signatures, public keys, and per-input witness references.

A high-level shape is:

```
pub struct Transaction {
    pub version: u16,
    pub inputs: Vec<TxInput>,
    pub outputs: Vec<TxOutput>,
    pub lock_time: u32,
    pub witness: Vec<u8>,
    pub tx_pow_nonce: u64,
    pub tx_pow_bits: u16,
}
```

The transaction model is deliberately narrow. There is no VM bytecode execution, no account nonce field, and no general script interpreter in the current consensus core. That reduces expressive power, but it also drastically reduces the number of consensus branches that can go wrong.

9. Transaction Lifecycle

A wallet begins by selecting spendable UTXOs, determining the payment amount, selecting a change address when needed, calculating a fee, constructing outputs, and building the unsigned transaction body. It then groups inputs by signing authority, computes the domain-separated Falcon digest for each signer group, signs, assembles witness data, fills witness commitment references, solves the transaction PoW, and produces canonical transaction bytes for submission.

Once the transaction reaches a node, the node parses it, rejects malformed or non-canonical encodings, checks structure and fee rules, optionally performs mempool policy checks, and admits it to the mempool only if the transaction is coherent and non-conflicting. When a miner later selects it, the transaction is revalidated inside block acceptance before any UTXO changes become final.

That lifecycle matters because every stage is a chance to drift. A wallet can build a transaction that is locally coherent but node-invalid. A mempool can admit something that later becomes stale. A miner can select a transaction that no longer has a spendable input. Atho's design expects those realities and makes the block validator the final authority.

10. Transaction Validation

Transaction validation in Atho is split between structure-only validation and context-aware validation. Context-free checks confirm that the object can exist at all: version, canonical encoding, non-empty input and output lists for normal spends, no duplicate inputs, output values above zero, output-count bounds, witness parse correctness, signature structure, fee floor compatibility, and transaction PoW correctness. Context-aware checks then confirm that each referenced UTXO exists, belongs to the signer, is on the same network, is mature if coinbase-derived, and that the transaction's total outputs do not exceed its total input value.

Rule	Reason	Consensus Risk if Missing
Non-empty input set for normal txs	Prevent malformed spends	Invalid tx acceptance
Non-empty outputs	Prevent nonsensical spends	Invalid tx acceptance
Duplicate input rejection	Stop same-tx double spend	Direct monetary failure
Exact UTXO lookup	Enforce real ownership	Double spend / theft
Falcon verification	Authenticate spend authority	Signature bypass
Network-bound spend and address checks	Stop cross-network replay	Replay acceptance
Output sum $\leq$ input sum	Preserve monetary integrity	Inflation
Fee / tx-PoW rule alignment	Spam resistance without drift	Policy/consensus mismatch

A key implementation detail is that the validator does not just trust a public key presented in the witness. It checks that the hash of that public key matches the canonical payment lock encoded in the spent UTXO, and then it checks the Falcon signature over the exact consensus digest. That two-step binding is what turns a bare signature into a valid spend authorization.

11. UTXO Model

The UTXO set is the source of truth for spendability. Historical blocks matter for replay, reindex, and auditing, but the current node answers the question “can this input be spent right now?” by consulting the active UTXO image, not by walking the whole chain every time. Each UTXO

entry encodes network, originating transaction ID, output index, amount, locking script, creation height, and whether the output came from coinbase logic.

Figure 4. UTXO state transition model.

```
for each input:
    lookup outpoint in live or staged UTXO set
    reject if missing, spent, wrong-network, wrong-owner, or immature
    remove from staged set
for each output:
    create new UTXO keyed by (txid, output_index)
fee = input_sum - output_sum
```

This model is simple enough to audit and strong enough to carry the whole chain. Failed transactions must not mutate it. Failed blocks must not mutate it. Reorgs must disconnect and reconnect exactly. Dirty restart verification must be able to prove that the persisted UTXO image still matches canonical block history. The recent internal work on Atho improved this significantly by replaying canonical history during dirty-start checks rather than trusting a tip snapshot alone.

12. Address and HPK System

Atho distinguishes between internal ownership locks and user-facing addresses. The user-facing address is a base56-encoded string with a network-specific visible prefix. Internally, ownership is represented by a canonical 32-byte payment digest or hashed public-key relationship. The network enum fixes prefix, internal HPK prefix, consensus ID, ports, and magic bytes so that objects from one network cannot be interpreted as valid on another without explicit code changes.

Address Part	Purpose	Consensus-Critical?
Visible prefix	Human-visible	network No by itself, but validation-critical at

Address Part	Purpose	Consensus-Critical?
	separation	ingress
Payment digest / locking script	Ownership lock	Yes
HPK / public-key digest	Bind Falcon pubkey to spend rights	Yes
Base56 encoding	User transport encoding	Indirectly; parsing must be deterministic

The important distinction is that user-facing addresses are convenience encodings. Consensus ultimately cares about the canonical locking bytes committed into the transaction and UTXO set.

13. Falcon-512 Signature System

Falcon-512 is Atho's spend-authorization primitive. In the codebase, the signature system is implemented in `atho-crypto/src/falcon.rs`, while the domain-separated message rules live in `atho-core/src/consensus/signatures.rs`. The wallet derives keys, constructs the transaction message digest, signs under the `AthoSignatureDomain::Transaction` domain, and serializes witness bytes. Validators reconstruct the same message and verify the signature against the provided public key, then verify that the public key actually corresponds to the spent lock.

The message design is more important than the algorithm brand name. A signature scheme only protects ownership if signing and verifying are bound to the same canonical bytes. Atho's digest rules intentionally mix the transaction signing digest with a network-specific consensus ID and genesis hash so signatures cannot be replayed as valid on a different network.


```

message = domain_separated_tx_digest(
    network.consensus_id,
    genesis_hash(network),
    tx.signing_digest_for_input_indexes(...)
)
signature = falcon_sign(secret_key, message)
valid = falcon_verify(public_key, message, signature)

```

Attack	Example	Required Defense	Required Test
Empty signature	Zero-length witness signature	Length + parse rejection	Empty signature test
Wrong message	Signature from another tx	Exact digest reconstruction	Cross-tx signature test
Wrong group	input Valid signature but wrong covered inputs	Input-index binding	Group mismatch test
Wrong network	Same tx body on another network	Consensus ID + genesis binding	Cross-network signature test
Wrong pubkey	Signature valid for another key	HPK/public-key binding	Wrong-key test
Cache poisoning	Cached valid result used for wrong bytes	Cache key includes exact object identity	Cache differential test

Falcon gives Atheris a post-quantum ownership story, but it also imposes implementation discipline. Signature parsing, length checks, public key handling, and message construction all become consensus-sensitive. The internal audits repeatedly treat mismatches here as mainnet-blocking.

14. Hashing System

Atho uses SHA3-family hashing for block identity, transaction identity, signature message prehashing, address/public-key digest work, Merkle roots, and witness commitments. In practice, SHA3-384 is the dominant consensus hash width in the chain, while SHA3-256 is also used in shorter commitment derivations such as witness reference tags.

Hash Use	Algorithm	Input	Output	Consensus-Critical?
Block hash	SHA3-384	Canonical header bytes	48 bytes	Yes
Txid	SHA3-384	Canonical transaction base bytes	48 bytes	Yes
Wtxid / witness commitment roles	SHA3-384	Full/canonical witness-aware bytes	48 bytes	Yes
Signature digests	SHA3-384	Domain-separated preimage	48 bytes	Yes
Short witness refs	SHA3-256	Txid + signature input index	+ 2 bytes excerpted	Yes, as part of witness reference scheme
Address / public-key digest	SHA3-family helper	Public key bytes	32 bytes used for lock	Yes

The critical engineering rule is that every hash must be defined over exactly one canonical encoding. Hashing a structurally equivalent but differently encoded object is a classic route to consensus splits.

15. Serialization and Canonical Encoding

Serialization is one of the most underestimated consensus surfaces in any blockchain. Atho's code generally treats canonical binary encodings as the authoritative representation for hashes, block files, and network interchange. That is the right stance. JSON is useful for human-facing RPC, but it is a dangerous source of truth for consensus because map ordering, whitespace, defaulted fields, and type coercions can change meaning across implementations.

The repository's recent hardening tightened full-transaction decoding so that truncated full encodings with omitted tx-PoW tails are rejected rather than silently defaulted. That is the correct direction. The rule Atho should preserve forever is:

```
decode(encode(x)) == x
encode(decode(bytes)) == canonical_bytes
hash(encode(x)) is identical across all honest nodes
```

Any non-canonical encoding accepted by one path and normalized differently by another is a potential split vector. That is why the internal audits put so much weight on parser fuzzing, trailing-byte rejection, duplicate-field rejection, and roundtrip tests.

16. Merkle Roots and Transaction Ordering

Merkle roots bind the ordered set of transaction identifiers to the header. If transaction ordering changes, the Merkle root changes. If the Merkle root in the header does not match the transactions in the block body, the validator rejects the block. The same logic applies to the witness root for the witness-aware commitment path. Odd leaf handling, duplicate txid behavior, and canonical transaction identity all matter because they affect whether two honest nodes compute the same root from the same block body.

The core rule is straightforward: a block is not just a bag of transactions. It is an ordered transaction list whose commitment must match the header exactly.

17. Proof-of-Work

Atho miners search the nonce space of a candidate header until the SHA3-384 hash of the canonical header bytes is less than or equal to the expected target. Validators recompute that hash independently. They do not trust the miner's reported result. They do not trust a claimed target from outside the active rules. They do not trust a block body merely because the header hashes low enough.

```
while true:
    header.nonce += 1
    digest = sha3_384(serialize_header(header))
    if digest <= target:
        return solved_block
```

The subtle point is that PoW only orders candidate blocks; it does not make invalid state transitions valid. Atho's validator correctly preserves that distinction by checking PoW and then still revalidating every transaction and the coinbase reward before accepting the block.

18. Difficulty Adjustment

The current codebase computes the next target from prior blocks and the candidate timestamp through `atho-core/src/consensus/pow.rs`. The exact formula is implementation-defined by that module, and this paper treats the code as the source of truth rather than inventing an external formula. The important properties are that validators must derive the same expected target from the same history, that target bounds are enforced, that timestamp influence is deterministic, and that network-specific rules remain isolated.

Where this paper is intentionally cautious is in not overspecifying behavior that should stay code-anchored. If a future published Atho protocol spec is created, the target formula should be reproduced from code and then regression-locked by independent test vectors.

19. Monetary Policy and Emissions

Atho's monetary policy is encoded in `atho-core/src/consensus/subsidy.rs`. The block subsidy is height-dependent, network-aware, and combined with valid transaction fees to determine the maximum allowed coinbase output value. All arithmetic is in atomic units. That is essential: consensus math cannot rely on floating-point arithmetic or human-readable decimal parsing.

Height Range	Reward	Notes
Genesis	Genesis-specific allocation	Defined by genesis state
Normal mining heights	<code>block_subsidy_atoms_for_network(network, height)</code>	Code is the source of truth
Future halving / tail boundaries	Implementation-dependent by subsidy module	Must remain deterministic

The recent property tests around cumulative issuance additivity and monotonicity are especially valuable. They do not prove the schedule is economically optimal, but they do prove that the implemented issuance helpers are internally coherent over large sampled height ranges.

20. Coinbase Transaction

The coinbase is the first transaction in every valid non-empty block and the only transaction allowed to create new coins. Atho's validator checks that there is exactly one coinbase, that it is first, that its special structure is respected, and that its output value does not exceed the exact subsidy plus valid fees in the block. Current hardening also tightened coinbase witness and tx-PoW shape so the coinbase cannot carry arbitrary extra witness bytes or non-zero tx-PoW fields.

That rigidity is good consensus design. Coinbase transactions should be more constrained than normal transactions, not less. They are the monetary entry point of the chain.

21. Fees, Dust, Weight, and Vsize

Atho computes fee as $\text{sum}(\text{inputs}) - \text{sum}(\text{outputs})$ for normal transactions. Transactions that overspend are invalid because the subtraction underflows in checked arithmetic. The code also maintains raw-size, weight, and virtual-size accounting for relay and block assembly. Mempool policy can reject some objects that historical block consensus might not, but the block validator must always recheck any consensus-critical fee and structural rule.

Dust handling sits on the boundary between policy and consensus. The repository's comments need to stay accurate about which dust limits are block-validity rules and which are relay-only rules. That distinction matters because nodes may be free to relay more strictly than they mine or accept in historical block contexts.

22. Mempool

The mempool is a policy buffer, not a source of truth. Atho's mempool tracks candidate transactions, detects conflicts, enforces policy limits, and prepares validated mining views. It must never become a substitute for block validation, and it must never mutate canonical chainstate. The correct sequence is: prevalidate for relay, store as unconfirmed candidate state, select for mining, then revalidate in the block path against the current chain tip.

A well-designed mempool improves operator experience and mining quality. A badly designed mempool can become a consensus footgun if validators start trusting cached outcomes too much. Atho's current architecture mostly gets this right, but differential testing between mempool acceptance and final block acceptance should keep expanding.

23. Miner

The miner path assembles a candidate block from the live tip and a filtered mempool view. It collects fee-bearing transactions that still fit block limits, ensures the block-level input set does not double-spend, calculates the coinbase reward as subsidy plus selected fees, finalizes witness commitment references, computes Merkle and witness roots, and exposes a candidate that a CPU or GPU solver can work on.

This is consensus-critical because if miner assembly calculates fees, ordering, commitments, or size differently from the validator, Atho can create self-invalid blocks. The safest pattern is exactly what the internal audits recommend: miner-built blocks should pass the same validator path as peer-received blocks. Atho already moves in this direction, and further miner-vs-validator differential tests would make that guarantee much stronger.

24. Block Propagation and Network Layer

Atho’s P2P layer handles peer handshakes, framed messages, transaction relay, block relay, header movement, and compact-block style reconstruction. The network layer is not allowed to decide validity. Its role is to move bytes, parse message envelopes, perform early rejection on obviously malformed or wrong-network traffic, and then pass candidate objects into node-side validation.

Network Message	Purpose	Validation Required	Risk
Version handshake	/ Establish compatibility	peer Protocol version and network checks	Peer spoofing / downgrade
Tx relay	Share unconfirmed transactions	Canonical decode + mempool policy + consensus prechecks	Spam / parse bugs

Network Message	Purpose	Validation Required	Risk
Block relay	Share candidate blocks	Canonical decode + full block validation	Invalid block acceptance
Headers / getheaders	Sync chain tips	Header validity and locator logic	Wrong-fork sync
Compact block	Reduce relay cost	Exact transaction identity reconstruction	Reconstruction ambiguity

One of the important recent fixes was moving compact-block short IDs away from plain txid ambiguity and toward the witness-aware committed identity. That is the kind of detail that can look small but directly affects whether nodes reconstruct the same block body.

25. Sync Strategy

Atho supports header and block synchronization, background validation controls, checkpoint-anchored sync settings, and snapshot bootstrap. Operationally, the node can accelerate startup and body download, but consensus must remain unchanged regardless of sync mode. That distinction is essential. A faster path is acceptable only if it produces the same chainstate a cold replay would have produced.

The repository's snapshot bootstrap feature is useful but still deserves caution. Hash pinning is helpful, but signed-distribution verification is stronger. The current audit position is that bootstrap acceleration is acceptable only if the node still has a path to full deterministic validation and restart integrity checking.

26. Chainstate and Storage

Atho stores block metadata, height indexes, transaction archive records, UTXOs, peer health data, runtime markers, and block payload archives through LMDB plus flat block files. The chainstate layer keeps a recent in-memory suffix and an undo stack for fast operations, while the database persists the canonical image.

Database	Stores	Consensus-Critical?	Corruption Risk	Recovery Method
Meta	Tip snapshot, runtime state, schema/version data	Yes	Wrong restart assumptions	Startup checks + quarantine
Blocks	Block records and chainwork	Yes	Wrong canonical history	Replay, reindex
Height index	Height -> hash	Yes	Wrong tip traversal	Replay verification
Transactions	Tx archive records	High	Explorer / tooling mismatch	Rebuild from blocks
UTXOs	Live spendable outputs	Critical	Direct balance corruption	Replay and quarantine
Block files	Canonical payload bytes	Critical	Incomplete history	Reindex / archive checks

The storage safety story improved meaningfully when the dirty-start recovery logic stopped trusting tip metadata alone and began replaying canonical block history to compare against persisted UTXOs. That is much closer to the standard a payment chain needs.

27. Reorgs and Fork Choice

Fork handling is where consensus design meets operational stress. Atho's chainstate tracks canonical blocks, candidate branches, and an undo stack. When a preferred branch is discovered, the node finds the fork point, checks reorg depth policy, compares cumulative work rather than raw height, disconnects the old suffix, reconnects the preferred suffix, and persists the resulting state. Recent work replaced a clone-heavy rollback strategy with an incremental journal, which reduces the memory spike of deep failure paths while preserving recoverability.

Figure 5. Reorg disconnect/reconnect flow.

```
find fork point
-> compare branch work
-> disconnect old canonical suffix
-> restore spent outputs from undo data
-> connect preferred branch block by block
-> on failure, roll back via journal or full validated rewrite
-> persist canonical tip and UTXO image
```

The important invariant is that replaying the final active chain from genesis must yield the same UTXO set as the live database after any completed reorg.

28. API/RPC Layer

Atho exposes read paths, transaction submission, block submission, mining-related commands, and operator controls through RPC and API layers. These surfaces are security-sensitive even when they are not themselves consensus rules, because any bypass here can feed malformed objects directly into the node.

Endpoint Class	Purpose	Can Mutate State?	Required Validation	Risk
Read-only explorer / status	Visibility into tip, balances, blocks	No	Safe serialization only	Low
Raw transaction submission	Local wallet / operator tx broadcast	Yes, via mempool only	Full parse + mempool validation	High
Raw block submission	Testing and local tooling	Yes, via block validator only	Full block validation	High
Mining template	Candidate construction	No direct chainstate mutation	Exact template assembly	Medium
Admin/debug	Operational maintenance	Potentially	Strict auth and network gating	High

The correct model is “all external input must enter through full validation.” Atho’s service layer mostly follows that model, and the production RPC hardening work around cookie auth and hashed credentials moves the operational side in the right direction.

29. Wallet Architecture

The wallet crate is responsible for deterministic key derivation, address generation, receive/change management, spend selection inputs, grouped Falcon signing, change handling, and transaction PoW solving. Wallets are not the consensus authority, but they are part of the security boundary because a wallet can create invalid or unsafe transactions long before a validator gets involved.

The Atho wallet already shows a useful discipline: it keeps transaction-building logic close to the same digest and witness-reference rules the validator expects. Even so, the audits correctly call

for more wallet-vs-node differential testing. A payment chain should be able to prove that a wallet-built transaction roundtrips through node parsing without changing txid, wtxid, witness references, fee semantics, or signature interpretation.

30. Mainnet, Testnet, and Regtest Separation

The code does not literally expose a `Regtest` enum variant; it exposes `Regnet`, plus a `Prunetest` network for low-difficulty storage and pruning exercises. That is an important implementation detail. User-facing materials may say “regtest-like,” but the code-level truth is Mainnet, Testnet, Regnet, and Prunetest.

Setting	Mainnet	Testnet	Regnet	Prunetest
Consensus ID	1	2	3	4
Visible address prefix	A	T	R	P
P2P port	56000	9100	9200	9300
RPC port	9010	9110	9210	9310
Role	Production	Public testing	Deterministic testing	local Pruning and storage stress

This separation is consensus-critical. If a block or transaction from one environment can be accepted by another, the chain is not safely partitioned.

31. Security Model

Atho assumes that honest full nodes validate independently, that the implemented hash and

signature primitives behave as expected, that a majority of effective hashpower is not permanently controlled by a successful attacker, that operators protect private keys and node hosts, and that storage corruption can be detected or quarantined rather than silently trusted. It does not assume that peers are honest, that wallets are bug-free, or that APIs are friendly by default.

The system explicitly tries to defend against invalid blocks, invalid transactions, wrong-network replay, malformed payloads, mempool conflicts, coinbase overclaims, and at least some forms of storage inconsistency. It does not automatically solve 51% attacks, endpoint compromise, key theft, user operational mistakes, or unsafe off-chain infrastructure choices. That is a normal limitation for a PoW UTXO network; what matters is stating it clearly.

32. Consensus Vulnerability Analysis

The strongest way to think about Atho's remaining risk is to separate core-validator correctness from proof-net maturity. The recent audits did not reproduce a live invalid-block acceptance path or an obvious inflation path in the normal node flow. That is good. But the remaining concerns still matter because they influence whether the implementation can be trusted under adversarial conditions or after future refactors.

Vulnerability	Cause	Impact	Defense	Required Test	Severity
Panic-prone startup wrappers	Convenience constructors	Process call crash	/ Fail-closed <code>try_*</code> paths	Startup error-path tests	High
	<code>panic!</code>	on operator			
	failure	confusion			
Unsigned snapshot distribution	Hash pin is not a Bootstrap full release trust signature model	Detached signature weakness	Detached signature verification	Good-signature bad-signature tests to	/ Medium High

Vulnerability	Cause	Impact	Defense	Required Test	Severity
Thin differential coverage	Wallet/miner/node may go unnoticed	Split or self-drift invalid outputs	Differential tests in CI	Wallet/node and miner/validator suites	High
Incomplete fuzz runtime gate	Build-only fuzzing misses runtime panics	Parser/DoS bugs survive	Run fuzz bins in CI	Short smoke + longer scheduled fuzz	High
Reorg crash-window gaps	Not every disconnect/reconnect boundary is fault-injected	Silent rollback bugs	Reorg failpoints	Disconnect/connect fault tests	High

The practical conclusion is that Atho is not currently a chain where the main danger is one giant obvious bug. The danger is that the proof net around a fairly solid consensus core is still thinner than the project should accept for a public mainnet.

33. Edge-Case Testing Plan

Atho needs explicit boundary tests for amounts, heights, timestamps, sizes, witness lengths, transaction counts, and reorg depth. The internal edge-case audit already lays out a strong matrix, and the project should treat that matrix as a release requirement rather than a wish list. The key principle is that every consensus rule needs valid, invalid, boundary, malformed, duplicate, empty, and restart-corruption coverage.

34. Adversarial Transaction Test Plan

The adversarial transaction suite should continue to exercise missing UTXOs, duplicate inputs,

wrong-network spends, wrong public keys, malformed signatures, immature coinbase attempts, overspending outputs, truncated encodings, and alternate witness shapes. The built-in `atho-adversarial` harness is a useful asset precisely because it gives the project one place to script broad matrix behavior rather than relying on only hand-written unit tests.

35. Adversarial Block Test Plan

Block adversarial tests should continue to cover no-coinbase, two-coinbase, coinbase-not-first, wrong Merkle root, wrong witness root, duplicate inputs across transactions, wrong height, wrong parent, bad timestamp, bad target, valid PoW with invalid body, and failed block atomicity. These are the tests that prove a node is actually a full validator rather than just a polite parser.

36. Fuzz Testing Plan

The fuzz target list is already meaningful: transaction decode, block decode, network-message decode, mempool admission, compact block reconstruction, RPC request parsing, witness parsing, sighash stability, and block validation. The gap is not target imagination; it is runtime enforcement. Fuzz targets that merely compile are helpful during refactors, but they do not provide the same safety as scheduled runtime execution that can actually discover panics or hangs.

37. Property-Based Testing Plan

The repository now has a real property-test layer for several critical invariants, including replay-from-genesis equivalence, UTXO apply/disconnect exactness, and reward schedule additivity. That is a meaningful upgrade. The next step is expanding property coverage toward failed-

validation atomicity, wallet/node serializer equivalence, and miner/validator candidate equivalence.

38. Differential Testing Plan

Differential testing is still one of the weaker areas in the proof net. The project should explicitly compare wallet-generated txids with node-parsed txids, miner-built block commitments with validator-recomputed commitments, live chainstate with reindexed chainstate, and explorer- or API-reported balances with raw UTXO-derived balances. In a small-chain codebase, differential tests often catch the exact kind of drift that unit tests miss.

39. Storage Crash Testing Plan

Crash testing should simulate failure before validation, after validation before DB write, during UTXO delete/create, during transaction index writes, during tip updates, and during reorg disconnect/reconnect. The desired post-restart invariant is simple: either the prior committed state survives or the new fully valid state survives, but no hybrid image is silently trusted.

40. Legacy and Bypass Audit

The repository should continue auditing for `legacy`, `fallback`, `debug`, `skip`, `mock`, `unsafe`, `trust`, `panic`, and similar markers. These strings are not automatically bad, but they are reliable indicators of places where temporary convenience can become accidental production behavior. The project already benefited from this kind of audit when `dev_seed_chainstate` was moved behind `test/devtools` gates.

41. Performance and Scalability

Atho's performance profile is shaped by Falcon signature cost, SHA3 hashing cost, UTXO database access, block-size and vsize accounting, mempool validation, and sync/replay behavior. The strongest performance wins so far come from avoiding unnecessary clones in explorer and read-only paths, reducing state duplication in reorg fallback logic, and keeping the validator deterministic rather than clever. Future optimization work should remain subordinate to correctness; high-throughput bugs are still bugs.

42. Code Examples

The most useful code examples in Atho are the ones that show real invariants rather than ornate syntax. The block validator in `atho-storage/src/validation.rs`, the UTXO apply/disconnect logic in `atho-storage/src/utxo.rs`, the miner template builder in `atho-node/src/mining.rs`, the signature-digest helpers in `atho-core/src/consensus/signatures.rs`, and the wallet grouped-signing flow in `atho-wallet/src/wallet.rs` are the core examples new contributors should study first.

43. Production Readiness Grading

The latest local phase audit rates Atho at roughly **7/10** for overall consensus security and **7/10** for overall mainnet readiness, with a **mainnet delay recommended** conclusion rather than a hard block. That means the codebase has passed the stage where a single obvious fatal consensus flaw is dominating the assessment, but it has not yet earned the kind of proof density that supports a casual launch posture.

Area	Grade	Explanation	Required Work
Overall consensus	7/10	Core validation path is solid; Differential, fuzz, crash	

Area	Grade	Explanation	Required Work
correctness		proof net still incomplete	coverage
Block validation	8/10	Strong staged validation and exact coinbase checks	More full-path negative tests
Transaction validation	8/10	Good structure/context split and witness checks	More malformed differential cases
UTXO accounting	8/10	Better rollback and replay tests	More persisted atomicity proofs
Monetary policy	8/10	Deterministic helper layer	More long-horizon schedule review
Falcon signature security	8/10	Strong domain-separated message rules	External crypto review and differential tests
Serialization safety	8/10	Canonical bytes, stricter decoders	More fuzzer runtime and alternate-encoding tests
Mempool safety	8/10	Good conflict handling and revalidation stance	More mempool/block drift tests
Miner correctness	8/10	Candidate assembly is careful	Explicit miner/validator differential gate
Reorg safety	8/10	Journalled rollback is much better	Additional disconnect/connect failpoints
Storage safety	8/10	Startup replay checks improved	Stronger signed snapshot and more crash lanes
Fuzz coverage	6/10	Targets exist, runtime gate still	Run fuzz in CI

Area	Grade	Explanation	Required Work
		weak	
Differential testing	5/10	Not enough cross-path comparisons yet	Add wallet/miner/node comparisons
Mainnet readiness	7/10	Close, but not yet boringly provable	Finish the hardening queue

44. Mainnet Blocking Conditions

Atho should treat the following as non-negotiable launch blockers: any path that can accept an invalid block, any path that can reject a valid block nondeterministically, any inflation route, any double-spend path, any coinbase maturity bypass, any signature bypass, any failed-validation state mutation, any reorg corruption, any serialization ambiguity that changes hashes across nodes, or any environment-leak path that lets wrong-network data enter mainnet.

The current internal judgment is that Atho no longer exhibits an obvious live example of those failures in the exercised validator path, but it still needs stronger proof that future changes and hostile inputs will not reintroduce them.

45. Mainnet Launch Checklist

Before launch, the team should require: full consensus unit tests; adversarial block and transaction suites; UTXO atomicity tests; coinbase and monetary-policy tests; serialization roundtrip tests; PoW and difficulty tests; network-separation tests; storage crash tests; fuzz runtime smoke in CI plus longer scheduled runs; property tests; differential tests; replay-vs-live chainstate equality checks; and a final pass confirming that no production path skips validation or mutates mainnet constants at runtime.

46. Operational Deployment

Operationally, Atheris should be run with network-specific data directories, explicit mining reward addresses on value-bearing networks, locked-down RPC exposure, backups of wallet material and configuration, monitoring of sync and peer health, and careful separation between public APIs and admin interfaces. Bootstrap nodes, explorers, and mobile wallet endpoints should be treated as infrastructure, not as trust anchors.

47. Governance and Upgrade Safety

Consensus changes are dangerous even when they look small. Version bits, activation heights, serialization changes, reward-schedule edits, difficulty changes, witness changes, or address-format changes can all split the network if rolled out casually. Atheris should therefore treat protocol changes as auditable releases with explicit migration notes, testnet rehearsal, release signing, operator communication, and rollback planning.

48. Current Risks and Open Issues

Risk	Severity	Area	Why It Matters	Required Fix
Thin differential coverage	High	Wallet/miner/node	Drift can hide until production	Add dedicated differential suites
Fuzz runtime not fully enforced	High	Parsers and protocol	Build-only checks miss crashes	CI fuzz execution
Snapshot distribution trust still limited	Medium to High	Bootstrap	Hash pin is not a full release	Detached signature verification and signature

Risk	Severity	Area	Why It Matters	Required Fix
			model	operator guidance
Panic-oriented convenience wrappers	High	Runtime startup	Crashes reduce fail-closed behavior	Prefer <code>try_*</code> everywhere in production
Reorg crash-window coverage incomplete	High	Chainstate	Silent rollback bugs are costly	More failpoints
External audit still pending	High	Project-wide	Independent review matters for mainnet	Commission external review

49. Glossary

Atho: The blockchain project analyzed in this paper.

UTXO: Unspent transaction output; the spendable state unit in Atho.

Txid: Transaction identifier derived from canonical transaction bytes.

Block hash: SHA3-384 digest of canonical header bytes.

Coinbase: Special first transaction in a block that creates subsidy and collects fees.

Falcon-512: The post-quantum signature primitive Atho uses for spend authorization.

SHA3-384 / SHA3-256: Hash functions used across block, transaction, and witness commitment roles.

HPK: Hashed public-key relationship used to bind public keys to ownership locks.

Mempool: Node-local set of validated but unconfirmed candidate transactions.

Chainstate: The active chain tip, recent canonical block suffix, and UTXO image.

Reorg: Replacement of the active canonical suffix by a higher-work competing branch.

Difficulty target: Threshold a block hash must satisfy to be valid.

Dust: An output amount so small that it is not economical or acceptable under current rules or

policy.

Canonical serialization: The single byte representation that consensus hashes and validators use.

Prunetest: Low-difficulty network for storage and pruning stress tests.

50. Conclusion

Atho is a serious payment-oriented UTXO blockchain implementation with a clear architectural personality. It keeps consensus anchored in a full-node validator, uses PoW for ordering, adopts Falcon-512 for post-quantum spend authorization, uses SHA3-family hashing throughout the protocol, and stores canonical state in an LMDB-backed chainstate that now has meaningfully better dirty-start verification and reorg rollback behavior than it did in earlier audit snapshots. Those are real strengths.

At the same time, a responsible mainnet judgment has to be earned by proof, not optimism. Atho's strongest current design choices are its deterministic UTXO model, explicit network separation, narrow consensus surface, exact coinbase reward enforcement, and growing library of regression and adversarial tests. Its most important remaining work is not inventing a new protocol feature. It is finishing the hardening around fuzz execution, differential testing, signed bootstrap trust, reorg crash-window fault coverage, and the removal of panic-oriented convenience paths from production-facing startup flow.

The engineering priority order is therefore clear: preserve consensus simplicity, expand proof around the consensus core, keep the wallet/miner/node paths locked together with differential tests, and continue failing closed under storage and restart stress. On the evidence available in the current repository and internal audits, Atho is **close but not yet comfortably mainnet-ready**. The right rating is **7/10 overall**, with **mainnet delay recommended** until the remaining proof-net gaps are closed.

References

Falcon Team. (2020). *Falcon: Fast-Fourier lattice-based compact signatures over NTRU* (Version 1.2). <https://falcon-sign.info/falcon.pdf>

Nakamoto, S. (2008). *Bitcoin: A peer-to-peer electronic cash system*. <https://bitcoin.org/bitcoin.pdf>

National Institute of Standards and Technology. (2015). *FIPS 202: SHA-3 standard: Permutation-based hash and extendable-output functions*. <https://doi.org/10.6028/NIST.FIPS.202>

Rust Project Developers. (2026). *Result in std::result - Rust*. <https://doc.rust-lang.org/stable/std/result/enum.Result.html>

Symas Corporation. (n.d.). *Symas LMDB technical information*. <https://www.symas.com/symas-lmdb-tech-info>

Reference needed: a future Atho protocol specification that freezes the code-level consensus rules into a versioned external standard.

Appendix A: Full Consensus Rule List

1. Blocks must decode from one canonical encoding and reject trailing or truncated garbage.
2. Block headers must commit to the exact merkle and witness roots validators expect.
3. Block network IDs must match the active network.
4. Block versions must be valid for the active ruleset height.
5. Block heights must match the expected parent-relative height.
6. Previous-block hashes must match the active tip or explicit fork context.
7. Timestamps must respect lower and upper bounds enforced by validation code.
8. Difficulty targets must equal the validator-derived expected target.

9. Header hashes must satisfy proof-of-work.
10. Blocks must contain exactly one coinbase and it must be first.
11. Duplicate transaction IDs must be rejected.
12. Duplicate inputs across a block must be rejected.
13. Normal transactions must pass context-free validation.
14. Normal transactions must pass contextual UTXO authorization.
15. Coinbase rewards must not exceed subsidy plus valid fees.
16. Failed validation must not mutate chainstate.
17. Reorgs must disconnect and reconnect exactly.
18. Replay from genesis must match live chainstate.
19. Mainnet, testnet, regnet, and prunetest constants must remain isolated.
20. Persisted corrupted state must be detected, quarantined, or rebuilt rather than silently trusted.

Appendix B: Full Test Matrix

Appendix B.1. Summary matrix

- Block tests: valid acceptance, wrong parent, wrong height, wrong timestamp, wrong target, bad PoW, bad merkle root, bad witness root, no coinbase, two coinbases, duplicate txids, duplicate inputs, overclaim, failed-block atomicity.
- Transaction tests: malformed encoding, no inputs, no outputs, duplicate inputs, missing UTXO, wrong owner, wrong network, immature coinbase, invalid Falcon signature, wrong public key, wrong HPK, overspend, dust, fee mismatch.
- UTXO tests: create, spend, failed spend rollback, failed block rollback, reorg disconnect/reconnect, replay equality, persisted corruption detection.
- Storage tests: commit fault injection, snapshot replacement rollback, startup consistency verification, quarantine on corrupt persisted state.

- Network tests: wrong-network messages, compact block reconstruction, handshake validation, protocol-version bounds.
- Wallet tests: deterministic derivation, grouped signing, oversized spend rejection, non-canonical locking script rejection, restore stability.

Appendix B.2. Incorporated internal edge-case and attack audit extract

Atho Full Edge-Case and Adversarial Consensus Test Audit

1. Executive Summary

Verdict

- **Overall edge-case coverage grade: 7/10**
- **Overall adversarial testing grade: 6/10**
- **Overall mainnet safety grade: 6/10**
- **Decision: MAINNET DELAY RECOMMENDED**

Why

The good news first: after the recent consensus hardening pass, I did **not** reproduce an active inflation bug, an obvious double-spend acceptance path, an invalid-block acceptance path, or a valid-block rejection bug in the current validator/miner/sync/storage path that I exercised locally.

The blunt part: Atho is now in the uncomfortable middle ground where the **core rules look materially stronger**, but the **proof machinery around them is still incomplete**. The biggest problems in this pass were not “the chain is obviously broken,” but rather:

1. the **fuzz gate is currently broken** and does not compile,
2. the built-in **adversarial runner has drifted from current consensus rules**,
3. there is **no property-based invariant suite**,
4. crash-fault injection is **too narrow** to prove full atomicity across all critical windows.

That is enough for me to say **do not call mainnet ready yet**, even though the consensus code

itself looks much healthier than before.

Evidence executed in this pass

Commands I ran:

- `cargo test -p atho-node --lib -- --test-threads=1 -> 198 passed`
- `cargo check --workspace -> passed`
- `python3 -m unittest tests.test_runtime_launcher -> 15 passed`
- `cargo check --manifest-path fuzz/Cargo.toml --all-targets -> failed`

Important failure from the fuzz build gate:

- `fuzz/fuzz_targets/common.rs:264` constructs `BlockHeader` without `founders_hash_sha3_384` and `founders_hash_sha3_512`, so the documented fuzz build check in `docs/testing.md:37-43` is currently false.

Top 25 Missing Edge-Case Tests

1. Exact PoW boundary test where `hash == target`
2. Timestamp exactly at maximum future drift
3. Timestamp one second beyond maximum future drift in full node sync flow
4. `u64::MAX` timestamp arithmetic safety
5. `u64::MAX` height arithmetic in chain selection helpers
6. Reorg replay property: replay-from-genesis equals post-reorg live state for random valid branches
7. Property test that failed block validation leaves UTXO set bit-identical
8. Property test that miner-produced candidate block always passes normal validator
9. LMDB value decoder fuzz target
10. Chainstate loader fuzz target from corrupted persisted bytes
11. Explorer snapshot loader fuzz target
12. Falcon public key parser fuzz target

13. Falcon signature parser fuzz target
14. Compact-block reconstruction differential test for tx variants with same txid but different malformed witness bytes
15. Crash injection after block DB write but before tx index update
16. Crash injection after tx index write but before UTXO updates
17. Crash injection after UTXO deletions but before UTXO insertions
18. Crash injection after UTXO insertions but before tip metadata update
19. Crash injection during reorg disconnect phase
20. Crash injection during reorg reconnect phase
21. Cross-network transaction replay tests at the API/raw-hex boundary for all networks
22. Exact fee-floor minus-one / equal / plus-one tests at block acceptance boundary
23. Multi-input same-signer-group / multi-signer-group randomized witness property tests
24. Snapshot bootstrap negative tests for wrong hash + wrong network + stale tip in one matrix
25. Full adversarial campaign CI lane with deterministic short-case budget

Top 25 Highest-Risk Failure Modes

1. Broken fuzz gate silently reduces parser coverage
2. Stale adversarial harness gives false confidence about hostile scenarios
3. Missing property tests leave invariant regressions hard to catch
4. Narrow crash fault injection can miss silent atomicity bugs
5. Hidden production mutation helper (`Node::dev_seed_chainstate`) could be reused unsafely in future tooling
6. Startup/load paths still include panic-oriented convenience entrypoints
7. Snapshot bootstrap is hash-pinned, not signed-distribution verified
8. Local clock skew can still temporarily reject valid future-near-tip blocks
9. Compact-block logic depends on mempool transaction identity remaining aligned with witness commitment rules
10. Differential drift between API-reported balances and live UTXO scan is only partially

covered

- 11.No proof that all malformed LMDB values fail closed under fuzz mutation
- 12.No proof that all malformed RPC payloads remain non-panicking under extended fuzz runs
- 13.No proof that all malformed P2P payloads remain non-panicking under extended fuzz runs
- 14.No randomized long-branch replay property across repeated reorgs
- 15.No long-duration soak of checkpoint-anchored sync under churn
- 16.No exact equality proof for PoW $\leq$ semantics
- 17.No randomized stress proving mempool/miner/validator agreement across large tx sets
- 18.No automated “reindex equals live state” invariant on every CI cycle
- 19.No automated “trusted snapshot load then full sync then reindex” equivalence test
- 20.Adversarial runner still uses non-canonical 4-byte locks in its base fixtures
- 21.Some operationally dangerous helper paths are only hidden by convention, not compile gating
- 22.Broken fuzz build means new header/serialization changes can outpace hostile-input coverage
- 23.No property test for coinbase reward equality across arbitrary fee combinations
- 24.No exhaustive malformed UTF-8 / invalid Base56 API regression matrix
- 25.No committed baseline for long-running fuzz or adversarial campaign durations

Top 25 Areas Attackers Would Target First

1. Full transaction decoder
2. Witness decoder
3. Block decoder
4. Compact-block reconstruction
5. P2P frame decoder
6. Network message decoder
7. RPC request decoder

8. Snapshot bootstrap loader
9. LMDB persisted value decoding
10. Reorg rollback path
11. Coinbase reward calculation boundary
12. Fee accounting overflow/underflow
13. Timestamp/future drift logic
14. Wrong-network replay surfaces
15. Falcon signature parsing and grouping
16. Witness input-reference binding
17. Mempool/miner candidate selection under concurrent churn
18. Admin/debug/developer helper surfaces
19. Crash recovery / unclean shutdown restart
20. Explorer/index snapshots as derived state
21. Pruning + restart + reorg combinations
22. Startup snapshot bootstrap with operator-supplied hash
23. Differential block template vs validator logic
24. Hidden fast paths / trusted modes / skip flags
25. Test harness drift that masks real regressions

Top 25 Fixes Required Before Mainnet

1. Repair the fuzz crate so `cargo check --manifest-path fuzz/Cargo.toml --all-targets` passes
2. Add fuzz targets for LMDB value decode and chainstate loader
3. Add fuzz targets for Falcon public key and signature byte parsing
4. Update `atho-adversarial` base fixtures to use canonical 32-byte payment locks
5. Replace stale adversarial “valid” fixtures that are now consensus-invalid
6. Add a deterministic short adversarial campaign CI target
7. Add `proptest` or `quickcheck` invariant suites for UTXO and replay correctness
8. Expand commit fault injection beyond `BeforeCommit`

9. Add crash-fault simulation around reorg disconnect/reconnect windows
10. Add exact PoW equality boundary tests
11. Add exact future-drift boundary tests in sync/runtime paths
12. Add replay-from-genesis equivalence tests after reorg
13. Add differential tests comparing mined candidate blocks to normal validator across randomized mempools
14. Add malformed LMDB restart regression fixtures
15. Add snapshot bootstrap corruption regression fixtures
16. Make hidden mutation helpers test-only or feature-gated
17. Prefer fallible startup paths in default local tooling where possible
18. Wire fuzz compile check into required CI
19. Add a nightly fuzz execution job, not just compile-check
20. Add a nightly adversarial runner job with bounded case count
21. Add a nightly replay/reindex equivalence job
22. Add clock-skew simulation tests
23. Add exact fee-floor boundary tests at block-context validation level
24. Add API regression tests for invalid UTF-8 / Base56 / trailing garbage
25. Refresh `docs/testing.md` only after the documented commands genuinely pass

Top 25 Regression Tests Required Before Mainnet

1. Fuzz crate build regression
2. Canonical adversarial fixture regression
3. PoW equality-at-target regression
4. Future-drift exact-boundary regression
5. Future-drift plus-one-second regression
6. Coinbase reward with computed fees only regression
7. Block metadata fee fields ignored by consensus regression
8. tx-PoW changes witness commitment regression
9. Compact-block short ID differentiates tx-PoW variants regression

- 10.Empty-witness coinbase must have zero tx-PoW regression
- 11.Malformed witness bytes alter commitment regression
- 12.Failed block leaves UTXO snapshot unchanged regression
- 13.Failed reorg leaves canonical chain unchanged regression
- 14.Crash-before-commit leaves snapshot unchanged regression
- 15.Crash-after-block-record-before-state-write regression
- 16.Snapshot hash mismatch fails closed regression
- 17.Wrong-network mining reward address rejected regression
- 18.Missing mainnet/testnet reward address rejected regression
- 19.Noncanonical raw transaction bytes rejected regression
- 20.Trailing-byte raw transaction bytes rejected regression
- 21.Wrong-network replay raw transaction rejected regression
- 22.Reindex result equals live state regression
- 23.Explorer snapshot rebuild equals live-derived state regression
- 24.Hidden dev seed helper absent from release feature set regression
- 25.Fuzz/common BlockHeader constructor stays in sync with header schema regression

2. Unknown Unknowns Review

Summary

This pass intentionally hunted for “things that are probably fine until they suddenly are not.”

The most important hidden-assumption findings were:

Finding	Why it is dangerous	Reachable?	Exploit / trigger	Fix	Proof test
Broken fuzz gate in Parser fuzz/fuzz_targets/common.rs:264	coverage exists only on paper if the fuzz	Yes	Header/schema changes land; current fuzz jobs	Update fixtures; current BlockHeader	fuzz CI for check fuzz crate build on

Finding	Why it is dangerous	Reachable?	Exploit / trigger	Fix	Proof test
	crate does not compile		silently stop being meaningful	schema	
atho-adversarial still models “valid” 4-byte locks at crates/atho-node/src/bin/atho-adversarial.rs:53-55, :339-358	Hostile campaign can misclassify invalid modern transactions/coinbases as valid fixtures	Yes		Rebuild adversarial fixtures canonical byte locks	Runner regression with one known-good canonical fixture
Many adversarial block cases use validate_block_with ut_pow (crates/atho-node/src/bin/atho-adversarial.rs:1037+))	Coverage misses full PoW acceptance path and can overstate block-level adversarial confidence	Yes		Split no-PoW tests CI full adversarial tests matrix and run both	
No repo-wide proptest quickcheck usage found	Invariant drift / can sneak past example-based tests	Yes	Replay UTXO / fee accounting regress over time	/ Add property suites UTXO, replay, subtly reward, encoding	Property test CI lane

Finding	Why it is dangerous	Reachable?	Exploit / trigger	Fix	Proof test
Crash fault injection only has CommitFaultPoint::BeforeCommit crates/atho-storage/src/db.rs:23 5-236	Atomicity is at only proven for one failure window	Test-only but coverage-critical	Bug appears after different partial-write phase	Add fault points a for delete/add/index /tip-update stages	Per-fault crash regression tests
Node::dev_seed_chain state is public in non-test builds at crates/atho-node/src/node.rs:462-474	Hidden mutation helpers can leak into operational tooling later	Not remotely reachable today	Future tool / Gate RPC #[cfg(test)] or exposes it	behind Release build explicit dev symbol/API feature regression	
Convenience startup paths panic on load failure (crates/atho-storage/src/chainstate.rs:179-182, crates/atho-node/src/node.rs:490-492)	Bad snapshot / corrupt local state becomes hard crash instead of surfaced error	Yes	Misconfigured snapshot or unrecoverable disk state	Prefer fallible startup in user-facing/default service constructors	Startup failure integration test
Testing docs claim fuzz build should pass (docs/testing.md:37-43) when it does not	Team can believe a gate exists when it is already broken	Yes	False release confidence	Keep docs and gates aligned; add CI required check	Docs command test in CI

3. Boundary Value Testing

Coverage status

Current boundary coverage is **good in core integer accounting and basic transaction structure, moderate in reorg/depth/checkpoint logic**, and **weak in extreme numeric / randomized boundary generation**.

Amount boundaries

Covered now

- zero-value output rejection:
crates/atho-storage/src/validation.rs:373-375
- dust rejection and exact dust-floor coverage:
crates/atho-storage/src/validation.rs:386-395, tests around :1736,
:1868
- fee floor calculations and examples:
crates/atho-core/src/consensus/tx_policy.rs:118, tests around
:514-735
- overflow-safe output sum: crates/atho-core/src/transaction.rs tests
around checked_output_value_atoms_rejects_overflow
- indefinite tail-emission policy explicitly tested:
crates/atho-core/src/consensus/subsidy.rs:115-137

Missing / should be added

- exact fee floor - 1 / == / + 1 tests in contextual block acceptance, not just tx
policy helpers
- u64::MAX input/output accumulation randomized tests
- “sum outputs greater than inputs by 1 atom” property tests across many shapes
- malformed numeric strings at API/raw-RPC boundaries
- no finite max-supply tests are intentionally replaced by “no cap remains” tests; that is

correct for Atho's current policy, but the report and CI should say so explicitly

Height boundaries

Covered now

- maturity and confirmation logic: `crates/atho-node/src/bin/atho-adversarial.rs:1334+`
- halving/tail-emission schedule tests:
`crates/atho-core/src/consensus/subsidy.rs`
- max reorg depth exact boundary / +1 rejection:
`crates/atho-storage/src/chainstate.rs:2527-2577`
- finalized checkpoint boundary tests:
`crates/atho-storage/src/chainstate.rs:2587+`

Missing / should be added

- randomized very-high-height reward and difficulty tests
- `u64::MAX` height arithmetic guards in replay helpers
- reorg across exact maturity boundary with mixed coinbase and non-coinbase spends

Time boundaries

Covered now

- timestamp zero rejected
- median-time-past floor enforced
- future-drift ceiling enforced:
`crates/atho-storage/src/validation.rs:1234-1238`
- exact future-drift rejection test added in
`crates/atho-storage/src/validation.rs:2485`

Missing / should be added

- exact equality test at `maximum_timestamp`

- one-second-under-boundary test in sync/block acceptance
- clock-skew / restart / NTP regression tests
- timestamp arithmetic overflow tests

Size boundaries

Covered now

- tx raw size / vsize checks
- block raw size / vsize / weight checks
- excessive tx-count allocation guard in block decode:
crates/atho-core/src/block.rs:419-423
- witness ref count / signature len / pubkey len parsing guards:
crates/atho-core/src/transaction.rs:262-329
- compact-block tx-count guard: crates/atho-p2p/src/protocol.rs:849+

Missing / should be added

- max-inputs exact boundary tests with randomized witness grouping
- one-byte-under / exact / one-byte-over block size cases in one parameterized matrix
- max-output-count exact boundary / +1 randomized tests across rule activations

4. Serialization Edge-Case Testing

What looks strong

- `Transaction::from_full_bytes()` is now strict and rejects truncated tx-PoW tails: crates/atho-core/src/transaction.rs:720-814
- trailing junk is rejected and tested in protocol fixtures and node service raw-tx tests
- block canonical decode rejects trailing garbage and oversized tx-count preallocation risk
- witness commitment now binds tx-PoW and malformed witness bytes

What is still missing

1. No property suite for `decode(encode(x)) == x` across randomized transaction and

block strategies

2. No explicit regression for endian confusion at the API/raw hex boundary
3. No dedicated malformed UTF-8 / overlong UTF-8 API request matrix
4. No differential test between wallet-produced tx bytes and node-produced tx bytes across many randomized spends
5. Fuzz build currently broken, which directly weakens serialization confidence

Current judgment

- **Serialization safety grade: 7/10**
- Core codec behavior looks disciplined now.
- The proof story is dragged down by the broken fuzz gate and absent property tests.

5. Transaction Edge-Case Testing

What is covered well

- no inputs / no outputs
- duplicate input rejection
- dust rejection
- zero-value output rejection
- fee floor checks
- tx size checks
- witness grouping shape checks
- wrong signer pubkey rejection
- wrong network ownership rejection
- missing UTXO rejection
- immature spend rejection
- tx-PoW exact-bits validation and network binding

Primary enforcement lives in `crates/atho-storage/src/validation.rs:360-720`.

What is still weak or missing

- randomized multi-signer grouped witness strategies
- block-level same-input-across-two-transactions fuzz/property testing
- malformed unlocking-script payload matrices at the API/raw-tx entrypoint
- explicit “same tx semantic content, different noncanonical encodings” rejection property
- malformed witness bytes under long fuzz runs

Current judgment

- **Transaction edge-case grade: 8/10**

This is one of the strongest areas right now.

6. Coinbase Edge-Case Testing

What improved

Coinbase shape is now much tighter:

- canonical payment lock required
- exactly one output required
- empty witness required
- `tx_pow_nonce == 0`
- `tx_pow_bits == 0`

See `crates/atho-storage/src/validation.rs:816-834`.

What is covered

- overclaim rejection
- duplicate coinbase rejection
- coinbase-not-first rejection
- wrong reward amount rejection
- legacy lock rejection
- maturity checks

What is still missing

- explicit underclaim test (accepted or rejected intentionally; document policy)
- reorg across coinbase maturity exact boundary with restored mempool transactions
- randomized fee combinations proving coinbase reward equals subsidy plus computed fees only

Current judgment

- **Coinbase edge-case grade: 8/10**

7. Block Edge-Case Testing

What is covered

- empty block rejection
- no coinbase rejection
- wrong network rejection
- wrong height rejection
- wrong parent rejection
- wrong merkle root rejection
- wrong witness root rejection
- duplicate txid rejection
- duplicate input across block rejection
- invalid tx inside block rejection
- future timestamp rejection
- invalid PoW rejection
- oversized block rejection

What is still missing

- `exact hash == target` acceptance
- exact max-size / max-weight boundary matrices
- malformed-body-after-valid-header randomized corpus
- crash-in-middle-of-contextual-validation failpoint proof

Current judgment

- **Block edge-case grade: 8/10**

8. UTXO and State Mutation Edge Cases

What is strong

- contextual validation uses an overlay before final mutation
- failed blocks do not mutate the live UTXO set in the tested paths
- chainstate reorg rollback has journaled restoration tests
- commit fault injection proves rollback on `BeforeCommit`

Notable tests:

- `crates/atho-storage/src/chainstate.rs:3134+`
- `crates/atho-storage/src/chainstate.rs:3547+`

What is still missing

- failpoints after UTXO deletion but before UTXO insertion
- failpoints after UTXO insertion but before metadata tip update
- failpoints around tx index writes and peer/address index updates
- proof that restart after each partial-write window equals clean replay-from-genesis

Current judgment

- **UTXO atomicity grade: 7/10**

The design is much better now, but the injected-fault matrix is still too narrow to claim “production-proven atomicity.”

9. Reorg and Fork Edge Cases

What is covered

- higher-work branch preferred over raw height
- deep reorg boundary enforcement

- finalized checkpoint conflict rejection
- rollback after candidate validation failure
- rollback after commit failure
- buffered side-branch and cross-peer reconstruction

This is now one of Aho's stronger areas.

What is still missing

- property-based replay equivalence after arbitrary valid branch switches
- crash during disconnect and reconnect phases
- mixed maturity-boundary and halving-boundary reorg matrices

Current judgment

- **Reorg safety grade: 8/10**

10. Mempool Edge-Case Testing

What is covered

- invalid consensus txs rejected
- wrong-network and invalid-signature paths rejected
- dust and fee-floor mismatches rejected
- mempool does not mutate chainstate
- mining view revalidates entries instead of trusting cached admission blindly
- invalid/stale unchecked entries are skipped at mining time

What is still missing

- longer parent/child randomized chains
- same semantic tx under alternate malformed encodings
- very-large mempool differential tests between mempool ordering and miner selection
- more explicit restore-after-reorg validity matrix

Current judgment

- **Mempool/miner alignment grade: 8/10**

11. Miner Edge-Case Testing

What is strong

- candidate blocks now require explicit payout address on mainnet/testnet
- candidate blocks compute fees and coinbase reward locally
- final mined block is still connected through normal validator logic

What is still missing

- large randomized mempool differential tests proving candidate block always passes validator
- more direct tests for mempool mutation during block assembly
- long-running CPU/GPU nonce result differential proof

Current judgment

- **Miner edge-case grade: 7/10**

12. PoW and Difficulty Edge Cases

What is covered

- target bounds
- retarget clamping
- network-specific stall reset behavior
- minimum next-block timestamp rules
- branch work comparison

What is still missing

- `exact hash == target` acceptance regression
- more explicit endianness abuse tests
- timestamp manipulation matrix around retarget boundary conditions

Current judgment

- **PoW/difficulty grade: 7/10**

13. Network Separation Edge Cases

What is strong

- network-scoped address decoding
- network-scoped tx signing digests
- wrong-network block rejection
- wrong-network UTXO ownership rejection
- separate storage roots per network
- finalized checkpoint logic is network-local

What is still missing

- fuller mixed raw-hex replay matrix across all three networks at external endpoints
- explicit regression that dev-only mining reward defaults never appear on mainnet/testnet

Current judgment

- **Network separation grade: 8/10**

14. Storage and Crash Edge-Case Testing

What is strong

- startup self-check clears stale commit journal
- recoverable local-state errors trigger quarantine/rebuild path
- schema mismatch fails closed
- raw block-file corruption is indexed safely through metadata

What is still missing

- multi-point crash injection coverage
- partial-write equivalence proof after every critical mutation window
- LMDB decode fuzzing

Current judgment

- **Storage crash safety grade: 6/10**

This is the biggest remaining proof gap after fuzzing/property coverage.

15. API/RPC Edge-Case Testing

What I found

External transaction and block inputs still route through normal validation paths. I did **not** find an API or RPC path that directly marks UTXOs spent or bypasses consensus checks.

Notable evidence:

- raw tx broadcast routes in `crates/atho-node/src/service.rs:375` and `:3001+`
- block submission route in `crates/atho-node/src/service.rs:420`
- API tests for malformed/hidden/oversized inputs passed in the full `atho-node --lib run`

Remaining concerns

- admin/debug operational surfaces still deserve separate deployment hardening review
- malformed external payload fuzzing exists conceptually, but the fuzz gate is broken

Current judgment

- **API/RPC consensus safety grade: 7/10**

16. Legacy and Dead Code Edge Cases

Keyword review table

Keyword	File	Function	Risk	Reachable ?	Required Action	
legacy	crates/atho-	canonical_p	Low	Yes	Keep	fail-

Keyword	File	Function	Risk	Reachable ?	Required Action
	storage/src/ validation.rs:466 -470	ayment_lock			closed; retain tests
skip_pow	crates/atho- storage/src/ validation.rs:940 -960, :1016-1025	validate_block_without_pow	Medium	Test/dev only practice	Keep out of production in entrypoints; document clearly
trusted	crates/atho- node/src/ node.rs:1497+ snapshot bootstrap paths	snapshot and bootstrap	Medium	Yes	Prefer signed snapshot metadata or stronger operator warnings
unchecked	crates/atho- node/src/ mempool.rs:656	insert_unchecked	Low	pub(crate) only	Keep internal; do not expose remotely
dev	crates/atho- node/src/ node.rs:462-474	dev_seed_chainstate	Medium	Yes	Gate behind test/dev feature
fast	crates/atho- node/src/ sync.rs:432-477,	fast download	Medium	Yes	Keep validation lag tests; add soak

Keyword	File	Function	Risk	Reachable ?	Required Action
	:2963+				
cache	crates/atho- node/src/ service.rs / explorer caches	explorer/ mempool caches	Low	Yes	Continue differential cache-vs-live tests
panic!	crates/atho- storage/src/ chainstate.rs:179 -182, crates/atho- node/src/node.rs: 490-492	load_or_new convenience paths	Medium	Yes	Prefer fallible startup in user- facing constructors
fallback	GPU/cookie/sync paths	multiple	Low	Yes	Keep explicit tests; mostly fail-safe so far
default	config/helpers	multiple	Low	Yes	Audit defaults when consensus schema changes

Bottom line

I did **not** find a live legacy consensus bypass comparable to the old non-32-byte lock risk from earlier audits. That specific issue appears closed now. The remaining “legacy/dead-code” concern is more about **stale test harnesses and helper surfaces** than about the main validator accepting old rules.

17. Fuzz Testing Plan

Existing fuzz targets

Current fuzz crate targets include:

- `p2p_frame_decode`
- `p2p_message_roundtrip`
- `tx_witness_parse`
- `tx_decode`
- `tx_roundtrip`
- `sighash`
- `block_decode`
- `block_template_decode`
- `block_validate`
- `mempool_admission`
- `compact_block_reconstruct`
- `network_message_decode`
- `rpc_request_decode`
- `address_decode`

Source: `fuzz/Cargo.toml`.

Gaps to add

1. Falcon public key decode
2. Falcon signature decode
3. LMDB UTXO value decode
4. LMDB chainstate snapshot decode
5. explorer snapshot decode
6. `config/rpcauth` parse
7. `cookie-auth` file parse

8. block-record file-location decode
9. reorg undo journal decode
- 10.API path parameter decode with invalid UTF-8 / Base56

Current status

- **Fuzz coverage grade: 4/10**

The target list is promising.

The actual gate is not.

18. Property-Based Testing Plan

Current status

Repo-wide search found **no proptest or quickcheck usage** under `crates/` or `fuzz/`.

Minimum property suite to add

1. UTXO transition equivalence for valid blocks
2. Failed block leaves UTXO set unchanged
3. Replay-from-genesis equals live state after arbitrary valid chain prefixes
4. Reorg replay equals active state after branch switch
5. Coinbase reward equals subsidy plus fees
6. `decode(encode(tx)) == tx`
7. `decode(encode(block)) == block`
8. `txid` stable across roundtrip
9. block hash stable across roundtrip
- 10.merkle root changes when tx order changes
- 11.witness root changes when committed witness/tx-PoW content changes
- 12.miner candidate block always passes normal validator

Current status

- **Property-test coverage grade: 2/10**

19. Differential Testing Plan

Current differential coverage I observed

- compact-block reconstruction vs full block path
- explorer incremental rebuild vs full rebuild
- chainstate reorg rollback vs preserved snapshot
- service `gettxoutsetinfo` vs listed UTXO stats
- branch work comparison vs reference logic in PoW tests

High-value differential tests still missing

1. miner fee calculation vs validator fee calculation on randomized mempools
2. wallet-built raw tx bytes vs node parse/re-serialize across random spends
3. CPU miner vs validator PoW equality corpus
4. chain replay from blocks vs persisted LMDB snapshot across randomized histories
5. API-reported balance vs direct UTXO scan over randomized address histories

20. Regression Test Requirements

Bug / Risk	Test Name	File	Fails	Passes	CI
			Before Fix?	After Fix?	Required ?
Broken header fixture	fuzz_block_header_schema_compiles	fuzz/fuzz_targets/common.rs	Yes	Not yet	Yes
Stale 4-byte fixtures	adversarial_valid_fixture_uses_canonical_locks	crates/atho-node/src/bin/atho-adversarial.rs	Yes	Not yet	Yes
Missing	PoW pow_accepts_h	crates/atho-core/	N/A	Not yet	Yes

Bug / Risk	Test Name	File	FAILS	PASSES	CI
			Before Fix?	After Fix?	Required ?
equality boundary proof	ash_equal_to_ target	src/consensus/pow.rs			
Missing drift exact-boundary proof	future_timest amp_at_exact_ drift_limit_i s_accepted	crates/atho- storage/src/ validation.rs	N/A	Not yet	Yes
Missing crash- window coverage	commit_fault_ after_utxo_de lete_restores _state	crates/atho- storage/src/db.rs chainstate.rs	/ N/A	Not yet	Yes
Hidden dev helper in release builds	release_build _omits_dev_se ed_chainstate	crates/atho-node/ src/node.rs	N/A	Not yet	Yes
Panic-oriented startup convenience path	default_servi ce_startup_su rfaces_recover able_load_er rors	crates/atho-node/ src/service.rs node.rs	/ N/A	Not yet	Yes
Missing replay equivalence proof	reorg_replay_ equals_fresh_ replay_proper ty	crates/atho- storage/src/ chainstate.rs	N/A	Not yet	Yes

21. CI / Mainnet Gate Requirements

Mainnet should not launch until all of these are true:

- `cargo check --workspace` passes
- `cargo test -p atho-node --lib -- --test-threads=1` passes
- core/storage/p2p consensus suites pass
- launcher tests pass
- fuzz crate compiles
- nightly fuzz execution runs for a minimum budget
- deterministic short adversarial campaign passes
- property tests pass
- replay/reindex equivalence job passes
- crash-fault injection matrix passes
- miner-produced blocks pass validator under randomized mempool scenarios
- no test harness is stale relative to current consensus schema

22. Severity Rating

Severity	File	Function / Area	Description	Attack / Failure Scenario	Consensus Impact	Fix
High	fuzz/fuzz_targets/common.rs:264	fuzz fixture construction	Fuzz gate does Parser not compile regressions after header slip schema because expansion	past fuzz CI is dead	Indirect serious	Update to Block
High	crates/atho-node/src/bin/atho-adversarial.rs:53-	adversarial harness fixtures	Built-in hostile runner non-canonical	Audit harness gives positives/false	Test-quality false risk	Rebuild with

Severity	File	Function / Area	Description	Attack / Failure Scenario	Consensus Impact	Fix
	55, :339-358		4-byte locks as “valid” base negatives data			32-byte
High	repo-wide absence	property/ invariant suite	No property-based proof for UTXO/replay/reward invariants	Subtle regressions survive example tests	Consensus regression risk	Add pr suite
High	crates/atho-storage/src/db.rs:235-236	commit fault injection	Only one crash fault exists	Silent corruption bug point in an untested mutation window	Potential consensus-state corruption	Add failpoint
Medium	crates/atho-node/src/node.rs:462-474	dev_seed_c hainstate	Hidden state-mutation helper compiled into non-test builds	Future debug tool or route exposes it unsafely	Local state corruption if misused	Gate to if feature
Medium	crates/atho-storage/src/chainstate.rs:179-182, crates/atho-node/src/node.rs:490	convenience startup	Panic-oriented startup instead fallible	Misconfigured path snapshot of bad local state error crashes	Availability or risk; fail-closed	Prefer startup default

Severity	File	Function / Area	Description	Attack / Failure Scenario	Consensus Impact	Fix
	-492		propagation	process		
Medium	docs/testing.md:37-43	testing documentation	Docs claim fuzz build should pass when it not currently fails	Teams believe a gate exists when it does not	Process risk	Keep lockste live co
Medium	crates/atho-node/src/bin/atho-adversarial.rs:1037+	block campaign validation mode	Many adversarial cases use validate_block_without_pow	Hostile block coverage skips full PoW acceptance path	Coverage risk	Split n full-Po matrice
Medium	repo-wide	long execution	Fuzz targets exist but no fuzz executed baseline proven in this pass	Malformed input bugs remain undiscovered	Coverage risk	Nightly executi
Low	crates/atho-node/src/service.rs:210+ :263+	constructor / split	Safer try_new exists, many helpers	Tests/tooling may exercise but harsher local path still	Low	Prefer try_r where p

Severity	File	Function / Area	Description	Attack / Failure Scenario	Consensus Impact	Fix
			use new	runtime		

23. Subsystem Grades

Subsystem	Grade	Mainnet Ready?	Missing Edge Cases	Biggest Risk	Required Fix
Edge-case coverage	7/10	No	property/fuzz/crash gaps	false confidence	repair gates and add invariants
Block validation edge cases	8/10	Mostly	exact equality, boundary matrices	PoW max-boundary drift	add boundary/property tests
Transaction validation edge cases	8/10	Mostly	randomized grouped strategies	subtle witness regressions	add property tests
Coinbase edge cases	8/10	Mostly	underclaim randomized matrix	and fee policy drift	add reward equality properties
Monetary policy edge cases	7/10	Mostly	randomized high-height checks	schedule drift	add property/boundary tests
UTXO atomicity	7/10	No	multi-point injection	crash silent write corruption	partial-extend failpoints

Subsystem	Grade	Mainnet Ready?	Missing Edge Cases		Biggest Risk	Required Fix		
Reorg safety	8/10	Mostly	arbitrary equivalence	replay	replay mismatch	add tests	property	replay
Serialization safety	7/10	No	fuzz build broken, no roundtrip property suite	parser regressions		fix fuzz properties	+	add
Falcon signature safety	7/10	Mostly	dedicated fuzzing	parser malformed edge cases	byte add	Falcon fuzz		
PoW/difficulty safety	7/10	Mostly	exact equality tests	target and skew	boundary ambiguity	add edge tests		
Mempool/miner alignment	8/10	Mostly	large differential	randomized tests	ordering/fee drift	add validator	miner-vs-	differential
Storage crash safety	6/10	No	partial-write matrix incomplete	silent restart corruption		expand fault injection		
Network separation	8/10	Mostly	fuller replay	external matrix	route-level drift	add raw input tests	cross-network	
Legacy bypass resistance	8/10	Mostly	stale harnesses		test-quality drift	update fixtures	adversarial	
Fuzz coverage	4/10	No	broken gate	compile	no real hostile parser confidence	repair fuzz	and	execute

Subsystem	Grade	Mainnet Ready?	Missing Edge Cases	Biggest Risk	Required Fix
Property coverage	test 2/10	No	absent	invariant regressions	add proptest
Regression coverage	test 7/10	Mostly	missing new gate regressions	future refactors	add targeted regressions
CI mainnet gate quality	5/10	No	broken fuzz gate, no nightly hostile false green runs		strengthen CI

24. Final Mainnet Decision

MAINNET DELAY RECOMMENDED

Why not blocked outright

In this pass I did **not** reproduce:

- invalid block acceptance
- valid block rejection due to an obvious deterministic bug
- inflation from coinbase overclaim
- double-spend acceptance
- immature coinbase spend acceptance
- signature bypass
- mempool/block consensus mismatch that lets bad state in

The recent consensus fixes materially improved the chain.

Why not ready

I still cannot sign “mainnet ready” because:

1. the **fuzz gate is currently broken**,

2. the built-in **adversarial runner is stale relative to current consensus rules**,
3. there is **no property-based invariant suite**,
4. crash/failpoint coverage is **not broad enough** to prove full atomicity.

That is not the same as “I found an active inflation bug.” It means the remaining risk is now **proof and coverage debt**, not obviously broken consensus logic. That is still enough to delay a production launch.

Before mainnet

Must fix before launch:

- fuzz crate compile failure
- stale adversarial harness fixtures
- missing property-based invariants
- narrow crash-fault matrix

Should fix before or immediately after:

- hide or feature-gate `dev_seed_chainstate`
- make default local service constructors prefer fallible startup paths
- add exact PoW equality and time-boundary regressions

25. Final Deliverables

This report provides:

1. `CONSENSUS_EDGE_CASE_AND_ATTACK_TEST_AUDIT.md`
2. edge-case coverage review by subsystem
3. adversarial transaction and block testing assessment
4. monetary and replay boundary review
5. UTXO/crash/reorg safety review
6. serialization/fuzz audit
7. Falcon/parser hostile-input review

8. API/RPC consensus-boundary review
9. legacy/dead-code keyword risk table
- 10.missing-test report
- 11.critical/high-priority finding list
- 12.subsystem grade table
- 13.mainnet launch decision

Closing note

The overall picture is encouraging: Atho is no longer failing because the core consensus code is obviously flimsy. The next tranche of work is about making the **proof that it stays correct** hard to fake:

- working fuzz gates,
- current adversarial fixtures,
- property invariants,
- broader crash-fault injection.

That is the difference between “looks strong in local review” and “I’m comfortable staking mainnet confidence on it.”

Appendix C: Full Code Module Map

Appendix C.1. Summary module map

- **atho-core**: protocol types, hashes, constants, consensus helpers.
- **atho-storage**: chainstate, LMDB, UTXO logic, validation, block files.
- **atho-node**: runtime, mempool, miner, sync, service, API, startup, dev tooling.
- **atho-p2p**: message framing, compact block logic, peer configuration, connection handling.

- `atho-wallet`: deterministic wallet, signing, keypool, datafile persistence.
- `atho-rpc`: RPC commands, request/response transport.
- `atho-qt`: desktop UX, settings, connection layer, wallet workflows.
- `fuzz/`: parser and validator fuzz targets.

Appendix C.2. Incorporated internal phase-by-phase security audit extract

Atho Consensus Security Vulnerability and Phase Test Audit

Audit date: 2026-05-18

1. Executive Summary

Scores

- Overall consensus security grade: **7/10**
- Overall mainnet-readiness grade: **7/10**
- Overall edge-case coverage grade: **8/10**
- Overall fuzz/property-test coverage grade: **6/10**
- Overall storage safety grade: **8/10**
- Overall reorg safety grade: **8/10**
- Final launch decision: **MAINNET DELAY RECOMMENDED**

What was tested in this refresh

Completed directly during this pass:

- `cargo check --workspace -> passed`
- `cargo check --manifest-path fuzz/Cargo.toml --all-targets -> passed`
- `python3 -m unittest tests.test_runtime_launcher -> 15 passed`

Previously verified clean in this same worktree before this refresh:

- `cargo test -p atho-node --lib -- --test-threads=1 -> 198 passed`
- targeted storage regressions:
- `dirty_restart_detects_persisted_utxo_corruption_and_quarantines_state`
- `full_snapshot_commit_faults_leave_prior_snapshot_intact`
- `commit_fault_injection_rolls_back_chainstate_mutation`
- `replay_from_genesis_matches_live_chainstate`
- `reward_schedule_stays_additive_and_monotonic`
- `apply_then_disconnect_restores_exact_utxo_image`

Rerun status during this refresh:

- I started fresh full reruns of `cargo test -p atho-node --lib -- --test-threads=1` and `cargo test -p atho-storage -- --test-threads=1`.
- Both reruns remained green through the service/sync and chainstate/property portions that matter most to this audit.
- I intentionally stopped those long reruns after the relevant evidence was re-established so I could finish the report without pretending they completed in this exact pass.

Bottom line

Current Atho is materially stronger than the earlier audited state.

I did **not** reproduce:

- an active invalid-block-acceptance path,
- an active signature-bypass path in the normal node flow,
- an obvious inflation bug,
- an obvious double-spend acceptance path,
- or a height-over-work fork choice bug.

The important shift is that several earlier blocking concerns are now closed:

1. the fuzz workspace now builds,
2. the adversarial harness now uses current canonical 32-byte locks,
3. there is now a real property-test layer for replay, rewards, and UTXO apply/disconnect,
4. startup recovery now replays canonical history and compares the persisted UTXO image,
5. storage failpoints now cover more than a single pre-commit hook,
6. `dev_seed_chainstate` is no longer a normal always-on production surface.

That said, Ato is still not "airtight."

The remaining reasons I am still recommending a delay are mostly **proof-net** and **operational-hardening** issues:

- fuzz targets compile, but I did not prove long-running fuzz execution in this pass,
- differential testing is still thin,
- `validate_block_without_pow` remains a public helper and the adversarial harness still leans on it heavily,
- startup/load convenience paths still panic in production-oriented wrappers,
- snapshot bootstrap remains hash-pinned, not signed-distribution verified,
- reorg/disconnect crash windows are better tested than before, but not fully exhausted.

What passed

- canonical transaction decoding rejects trailing bytes and truncated encodings,
- fee metadata is no longer consensus-authoritative,
- tx witness commitment now binds tx-PoW fields,
- compact block short IDs are no longer plain-txid ambiguous,
- coinbase witness and tx-PoW shape are stricter,
- full restart integrity now checks replayed UTXOs against persisted LMDB state,
- new property tests enforce replay/reward/UTXO invariants,
- new fault-injection tests show snapshot replacement preserves the old committed image on mid-commit failure.

What was not fully proven in this pass

- long-duration fuzz execution,
- full end-to-end adversarial harness runtime,
- miner vs validator randomized differential testing,
- wallet vs node txid/serialization differential testing,
- reorg crash windows at every disconnect/reconnect substep,
- external independent review.

Top 25 consensus-breaking risks

1. Public `validate_block_without_pow` helper remains available and heavily used in diagnostics.
2. `Node::load_or_new` still panics on snapshot bootstrap failure.
3. `Chainstate::load_or_new` still panics on persistent-state load failure.
4. Snapshot bootstrap is hash-pinned, not signed by a release trust root.
5. Fuzz targets compile, but runtime fuzz coverage is not enforced by CI.
6. Differential testing between wallet, miner, validator, and explorer is still thin.
7. Reorg crash testing does not yet cover every disconnect/reconnect write boundary.
8. No persisted UTXO state root/commitment is stored for cheap startup attestation.
9. Local clock skew can still make future-timestamp enforcement temporarily divergent operationally.
10. Adversarial harness still uses no-PoW validation for many matrix cases.
11. No dedicated fuzz target exists for LMDB value decoding.
12. No dedicated fuzz target exists for undo-data decoding.
13. No dedicated fuzz target exists for chainstate loader recovery paths.
14. No dedicated property test proves failed block validation leaves persisted state bit-identical.
15. No randomized differential harness proves reindex state equals live state across many histories.
16. No wallet-vs-node transaction serializer differential suite exists.

- 17.No miner-vs-validator candidate differential suite exists under mempool churn.
- 18.Explorer/API read views lack direct corruption-differential checks against canonical chainstate.
- 19.Public startup wrappers prefer panic over fail-closed error returns.
- 20.Devtools feature still exists and can be enabled in custom builds.
- 21.RPC/API malformed raw block submission matrix is not exhaustive.
- 22.Network replay negative coverage exists, but not yet across every external ingress surface.
- 23.No signed-release proof exists for bootstrap snapshot bundles.
- 24.No CI gate currently runs fuzz, property, crash, and adversarial lanes together.
- 25.No external audit has yet validated the same fixes independently.

Top 25 security vulnerabilities or dangerous gaps

1. **High:** crates/atho-storage/src/validation.rs exposes validate_block_without_pow.
2. **High:** crates/atho-node/src/node.rs load_or_new panics on bootstrap failure.
3. **High:** crates/atho-storage/src/chainstate.rs load_or_new panics on load failure.
4. **High:** unsigned snapshot bootstrap trust model in crates/atho-node/src/node.rs.
5. **High:** missing runtime fuzz evidence for parser/state loaders.
6. **High:** missing LMDB decoder fuzz target.
7. **High:** missing undo-data fuzz target.
8. **High:** missing randomized miner/validator differential suite.
9. **Medium:** future timestamp ceiling depends on local wall clock.
10. **Medium:** no persisted state root for startup attestation.
11. **Medium:** no fully automated explorer-vs-chainstate differential check.
12. **Medium:** reorg crash windows are not fully fault-injected.

- 13.**Medium:** no explicit CI gate for reindex/live-state equivalence.
- 14.**Medium:** no proof that malformed stored transaction bytes are fuzzed end-to-end.
- 15.**Medium:** no dedicated public-key parser fuzz target beyond signature-path coverage.
- 16.**Medium:** no long random fork forest differential test.
- 17.**Medium:** no explicit block hash == target integration test recorded in this pass.
- 18.**Medium:** no end-to-end wrong-network raw block RPC matrix recorded in this pass.
- 19.**Medium:** no end-to-end wrong-network raw tx RPC matrix recorded in this pass.
- 20.**Medium:** some library-friendly convenience paths still panic instead of returning typed errors.
- 21.**Low:** devtools-only helpers remain in the tree and need release discipline.
- 22.**Low:** command metadata policy/permission declarations still risk drifting from enforcement.
- 23.**Low:** snapshot bootstrap operator error can still produce confusing startup behavior even if it fails closed.
- 24.**Low:** runtime launcher GPU fallback warnings are operational noise, not consensus risk.
- 25.**Low:** audit automation is still weaker than the manual proof now present in tests.

Top 25 missing tests

1. LMDB value decoder fuzz target
2. undo-data fuzz target
3. chainstate loader fuzz target
4. long-run block parser fuzz campaign
5. long-run tx parser fuzz campaign
6. signature/public-key parser fuzz campaign
7. randomized failed-block atomicity property test
8. randomized reorg replay equality property test beyond short histories
9. miner-vs-validator differential test under mempool churn
10. wallet-vs-node txid differential test
11. wallet-vs-node serialization differential test

- 12.explorer balance vs UTXO scan differential test
- 13.reindex vs live state randomized differential test
- 14.exact hash == target integration test
- 15.exact future drift boundary integration test
- 16.reorg disconnect crash fault injection
- 17.reorg reconnect crash fault injection
- 18.tx-index partial-write crash test
- 19.undo metadata corruption recovery test
- 20.malformed stored transaction record recovery test
- 21.raw wrong-network block RPC matrix
- 22.raw wrong-network tx RPC matrix
- 23.snapshot signature verification test once signed bundles exist
- 24.CI fuzz smoke execution gate
- 25.CI property/differential/crash combined gate

Top 25 edge cases not fully covered

1. exact PoW equality boundary in node ingress
2. exact future timestamp ceiling under skewed local clocks
3. malformed persisted undo records
4. malformed persisted tx index records
5. malformed persisted address index records
6. partially persisted side-chain metadata during reorg
7. randomized mempool churn during candidate assembly
8. wallet-generated edge-case witness variants compared to node parsing
9. explorer reads during in-progress recovery after quarantine
- 10.snapshot bootstrap with archive mismatch but valid hash file
- 11.very large reindex history differential replay
- 12.wrong-network raw RPC payloads across every method alias
- 13.serialized legacy/current mixed object fuzzing

- 14.compact block reconstruction with pathological short-id collisions
- 15.state loader behavior under multiple simultaneous corrupted datasets
- 16.parallel signature verification under adversarially large blocks
- 17.replay after cross-network archive contamination attempt
- 18.malformed canonical block body with valid header and valid archive wrapper
- 19.future timestamp attack sequence across retarget boundaries
- 20.crash after block archive write but before height index rebuild on reorg
- 21.repeated dirty restarts after quarantine rotation
- 22.startup after partial snapshot replacement plus stale journal
- 23.API/public explorer response parity immediately after deep reorg
- 24.duplicated external submission over mixed P2P and RPC ingress
- 25.mainnet build accidentally including devtools

Top 25 fixes required before mainnet

1. Make fuzz execution, not just fuzz compile, mandatory in CI.
2. Add LMDB value decoder fuzz target.
3. Add undo-data fuzz target.
4. Add chainstate loader fuzz target.
5. Add randomized miner-vs-validator differential testing.
6. Add wallet-vs-node txid and serialization differential tests.
7. Add reindex-vs-live-state randomized differential tests.
8. Add reorg disconnect failpoints.
9. Add reorg reconnect failpoints.
- 10.Reduce visibility/scope of `validate_block_without_pow`.
- 11.Convert `load_or_new` panic wrappers to fail-closed production returns or isolate them to binaries.
- 12.Add signed snapshot bundle verification if snapshot bootstrap remains supported.
- 13.Add a persisted state root or equivalent cheap integrity commitment.
- 14.Add exact PoW boundary integration coverage.

15. Add exact future timestamp boundary integration coverage.
16. Add malformed persisted record recovery tests for tx/undo/address datasets.
17. Add raw wrong-network block RPC tests.
18. Add raw wrong-network tx RPC tests.
19. Add explorer-vs-chainstate differential checks.
20. Add long-history randomized replay property tests.
21. Add failed-block atomicity property tests at persisted storage level.
22. Add a mainnet CI gate that includes fuzz, property, differential, and crash lanes.
23. Add release automation that forbids `devtools` in production artifacts.
24. Add a second independent manual review of consensus serialization and storage recovery.
25. Re-run this audit after the above lands.

Top 25 tests required before mainnet

1. full fuzz smoke run for every target
2. LMDB decoder fuzz run
3. undo-data fuzz run
4. chainstate loader fuzz run
5. tx parser fuzz run
6. block parser fuzz run
7. sighash fuzz run
8. address decode fuzz run
9. randomized reindex/live differential test
10. randomized miner/validator differential test
11. randomized wallet/node txid differential test
12. reorg disconnect crash test
13. reorg reconnect crash test
14. tx index partial-write crash test
15. malformed stored tx record recovery test

- 16.malformed stored undo record recovery test
- 17.exact PoW equality test
- 18.exact future drift ceiling test
- 19.wrong-network raw block RPC test
- 20.wrong-network raw tx RPC test
- 21.explorer balance differential test
- 22.failed-block persisted atomicity property test
- 23.long random replay-from-genesis property test
- 24.signed snapshot verification test
- 25.release artifact no-devtools assertion test

Top 10 areas needing a second audit

1. startup recovery and quarantine flow
2. snapshot bootstrap trust model
3. differential behavior between miner, wallet, validator, and explorer
4. fuzz target effectiveness after compile fixes
5. reorg crash windows
6. compact block reconstruction under adversarial short-id scenarios
7. full RPC malformed ingress matrix
8. release artifact feature gating
9. long-history replay equivalence
- 10.storage corruption behavior outside the UTXO dataset

Top 10 areas most likely to cause chain splits

1. serialization regressions
2. future timestamp logic under skewed clocks
3. reorg rollback/replay regressions
4. compact block reconstruction drift
5. wallet/node serialization drift
6. miner/validator fee or ordering drift

7. public no-PoW helper misuse
8. corrupted persisted state trusted differently across nodes
9. snapshot bootstrap trust mistakes
10. untested hard fork activation changes

Top 10 areas most likely to cause inflation

1. coinbase exact reward math
2. fee summation drift
3. failed-block partial state mutation
4. reorg rollback mistakes
5. output sum overflow regressions
6. corrupted UTXO values trusted as spendable
7. replay/reindex drift
8. snapshot bootstrap of wrong state
9. legacy format regression
10. wallet/node value-serialization drift

Top 10 areas most likely to cause state corruption

1. mid-commit crash windows
2. reorg disconnect/reconnect persistence
3. partially corrupted LMDB datasets outside UTXOs
4. startup recovery wrapper panics
5. archive write plus metadata drift
6. stale undo data
7. repeated dirty restarts
8. reindex/live mismatch
9. devtools misuse in custom builds
10. snapshot replacement interruption

2. Full Consensus Phase Map

Phase	Input	File/Module	Function/Class	Consensus Role	Failure Impact	Risk Level
Network phase	input framed peer bytes	crates/atho-node/src/tcp_p2p.rs	validated_payload_len, read loop	reject oversized/wrong-g-magic traffic before decode	DoS / wrong-network ingress	High
API/RPC phase	input HTTP/RPC payloads	crates/atho-node/src/service.rs, crates/atho-node/src/apis.rs	command dispatch, handlers	external route consensus object ingress	invalid tx/block injection	Critical
Transaction decoding phase	raw tx bytes	crates/atho-core/src/transaction.rs	from_full_bytes	build canonical object	divergent tx meaning	tx Critical
Transaction canonicalization phase	tx object	crates/atho-core/src/transaction.rs	base_bytes, full_bytes	single canonical shape	byte split / txid drift	Critical
Transaction validation phase	tx context	crates/atho-storage/src/validation.rs	validate_transaction*	reject spends	invalid inflation / reject-valid	Critical
Signature verification	tx witness	crates/atho-core/src/	digest verifier builders, Falcon helpers	spend signature authorization	bypass	Critical

Phase	Input	File/Module	Function/Class	Consensus Role	Failure Impact	Risk Level
phase		consensus/ signatures.rs, crates/atho- storage/src/val- idation.rs				
UTXO phase	lookup input refs	crates/atho- storage/src/ validation.rs, crates/atho- storage/src/ut- xo.rs	prepared validation, UtxoSet access	resolve targets	spend false false reject	accept / Critical
UTXO authorization phase	spend prepared input context	crates/atho- storage/src/ validation.rs	ownership, maturity, checks	value authorize spendability	double immature spend	spend / Critical
Fee phase	calculation inputs + outputs	crates/atho- storage/src/ validation.rs	fee math helpers, coinbase checks	exact accounting	fee inflation valid	/ reject- Critical
Mempool admission phase	candidate tx	crates/atho- node/src/ mempool.rs	mempool validators/admiss- ion	pre-screen relay candidates	mempool/block mismatch	High
Miner template phase	block mempool snapshot	crates/atho- node/src/ node.rs, crates/atho-	candidate construction	block assemble valid mineable block	miner/validator drift	Critical

Phase	Input	File/Module	Function/Class	Consensus Role	Failure	Impact	Risk Level
		node/src/mining.rs					
Coinbase construction phase	height fees	+ crates/athoreward node/src/mining.rs	output exact construction + fee claim	subsidy inflation			Critical
Block serialization phase	block object	crates/athoreward core/src/block.rs	canonical_bytes, from_canonical_bytes	canonical block bytes	split / corruption	archive	Critical
Header construction phase	candidate block data	crates/athoreward core/src/block.rs, crates/athoreward node/src/mining.rs	header fields, nonce target	PoW identity	invalid hash	block	Critical
Merkle calculation phase	root tx list	crates/athoreward core/src/block.rs	merkle_root, witness_root	commit body	block invalid-body acceptance		Critical
Proof-of-work phase	header bytes	crates/athoreward core/src/consensus/ pow.rs	meets_target, target schedule	work validation	accept header	invalid	Critical
Block	receipt decoded	crates/athoreward	buffered/full	sequence	and wrong	chain	High

Phase		Input	File/Module	Function/Class	Consensus Role	Failure Impact	Risk Level
phase		inbound block	node/src/sync.rs	block handlers	gating	attachment	
Block phase	decoding raw bytes	block	crates/atho-core/src/block.rs	from_canonical_bytes	object reconstruction	parser divergence	Critical
Header validation phase	block header chain context	+	crates/atho-storage/src/validation.rs	contextual header precheck	timestamp parent / target	/ reject-valid accept-invalid	/ Critical
Block validation phase	body block tx set		crates/atho-storage/src/validation.rs	validate_block	full consensus body rules	invalid acceptance	block Critical
Transaction revalidation inside phase	block txs block		crates/atho-storage/src/validation.rs	prepared block tx collection, parallel signature verify	do not trust mempool	invalid tx in block	Critical
Coinbase reward validation phase	coinbase tx		crates/atho-storage/src/validation.rs	coinbase-specific rules	exact reward enforcement	inflation	Critical
Monetary policy validation phase	height fees	+	crates/atho-core/src/subsidy.rs, crates/atho-	subsidy cumulative issuance checks	and supply discipline	inflation	Critical

Phase	Input		File/Module	Function/Class	Consensus Role	Failure Impact			Risk Level
UTXO transition phase	state valid block		storage/src/validation.rs						
			crates/atho-storage/src/chainstate.rs, crates/atho-storage/src/utxo.rs	connect_block, apply_block	spend/create UTXOs	balance corruption			Critical
Block phase	storage validated	state delta	crates/atho-storage/src/db.rs	commit_chainstate, archive append	atomic persistence	dirty state			Critical
Chain tip update phase	committed snapshot		crates/atho-storage/src/db.rs, crates/atho-storage/src/chainstate.rs	snapshot/tip persistence	canonical identity	tip split / restart	dirty		Critical
Reorg phase	better branch		crates/atho-storage/src/chainstate.rs	branch selection / rollback journal	choose highest work	wrong fork / corruption	valid		Critical

Phase	Input	File/Module	Function/Class	Consensus Role	Failure Impact	Risk Level
Crash recovery phase	dirty restart	crates/atho-storage/src/db.rs	runtime markers, journal cleanup, consistency replay	detect/reject corrupted persisted state	silent corruption	Critical
Reindex phase	persisted chain history	crates/atho-storage/src/chainstate.rs, crates/atho-node/src/apis.rs	reload/replay/quarantine paths	rebuild canonical state	wrong balances	High
Snapshot/state verification phase	snapshot bundle persisted DB	crates/atho-node/src/node.rs, crates/atho-storage/src/db.rs	bootstrap and persisted consistency	trust root and integrity	load wrong state	Critical
Explorer/API reporting phase	canonical state reads	crates/atho-explorer.rs, crates/atho-node/src/service.rs	explorer and views	index wrong public chain view	operator confusion balance drift	/ Medium
Wallet/node compatibility	wallet-built	crates/atho-wallet/src/	tx construction and parsing	reject-valid	/ High	Partial

Phase	Input	File/Module	Function/Class	Consensus Role	Failure Impact	Risk Level
phase	txs	wallet.rs, crates/atho- core/src/trans action.rs	parity	txid drift		

3. Threat Model

Attacker	Attack goal	Targeted subsystem	Current defense	Missing defense	Required tests/fixes	Severity
Malicious wallet	craft malformed or misleading txs	tx parser, tx validator, wallet/node compatibility	canonical decode, signature checks, network-bound locks	wallet/node differential tests	wallet-vs-node txid and serialization differential suite	High
Malicious miner	mine invalid block on valid-coinbase looking fork	block validator, coinbase rules, PoW checks	full revalidation, exact coinbase math	block broader miner/validator differential coverage	randomized candidate-vs-validator tests	Critical
Malicious peer	flood malformed blocks/txs or wrong-network payloads	tcp/p2p/ protocol/sync	payload caps, network magic, route validation	more malformed ingress matrices	fuzz + raw ingress negative tests	High
Corrupted	load	bad DB	open, quarantine,	no persisted	corrupted record	Critical

Attacker	Attack goal	Targeted subsystem	Current defense	Missing defense	Required tests/fixes	Severity
local node	LMDB partial archives	or recovery, replay	replayed UTXO verification	state root	matrix + state root	
Broken but honest node	local clock skew or panic path	contextual timestamp, load wrappers	future timestamp ceiling, <code>try_*</code> loaders	panic wrappers remain	fail-closed production wrappers	High
Malicious API client	submit malformed tx/block or hidden routes	raw service/API ingress	auth, hidden-gating, canonical hex decode	full wrong-network raw ingress matrices	test expansion	High
Disk corruption scenario	mutate UTXOs indices	stored storage or open/recovery	replayed UTXO comparison, cross-network rejection	more dataset-specific corruption coverage	LMDB/undo/tx record corruption tests	Critical
Reorg attacker	force deep corrupt rollback	invalid reorg or selection, undo/journal	work-based selection, depth, journal	reorg max window rollback coverage incomplete	crash disconnect/reconnect failpoints	Critical
Serialization attacker	exploit alternate encodings	tx/block codec, witness commitment	strict canonical long-running decode, legacy fuzz rejection	long-running + diff coverage	parser fuzz differential suites	+ Critical
Network	inject wrong- network	network IDs, network-specific	wider external raw RPC/API		wrong- Critical	

Attacker	Attack goal	Targeted subsystem	Current defense	Missing defense	Required tests/fixes	Severity
replay attacker	network objects/state	genesis, UTXO network tags	headers, network replay rejection	cross-ingress coverage	network matrices	

4. Phase 1: Network and API Input Security

Status: **stronger than before, but still not exhaustive**

What I verified:

- public API routes reject hidden write paths from browser origins,
- path/query/body validation fails closed for malformed explorer/API inputs,
- raw transaction hex parsing rejects trailing bytes and non-canonical encodings,
- P2P input framing checks size and magic before deeper decode.

Remaining gaps:

- I did not prove every raw wrong-network block/tx ingress surface across P2P, RPC, and API in one matrix.
- I did not run long fuzz execution for protocol decoders in this pass.

Network/API Input Security Grade: **8/10**

5. Phase 2: Transaction Parsing and Canonicalization

Status: **good and much stricter than the earlier audited state**

Verified:

- `Transaction::from_full_bytes` is strict and rejects trailing bytes,
- canonical bytes are stable through encode/decode tests,
- legacy lock formats are rejected in validation,

- parser pre-allocation hardening is present.

Remaining gaps:

- no dedicated long-run fuzz campaign was executed in this pass,
- no wallet-vs-node differential serializer suite yet exists.

Transaction Parsing and Canonicalization Grade: **8/10**

6. Phase 3: Transaction ID and Hashing Security

Status: **materially improved**

Key positive change:

- tx-PoW ambiguity was addressed by binding tx-PoW fields through the witness commitment path, so block commitments are no longer blind to them even though txid intentionally remains witness-excluding.

Verified:

- tx canonical decode/encode tests exist,
- compact block reconstruction no longer relies on plain txid identity for this issue class,
- service/API tests cover non-canonical raw tx rejection.

Remaining gaps:

- no wallet/node differential txid suite,
- no explorer/node tx identity differential suite.

Transaction Hashing Grade: **8/10**

7. Phase 4: Falcon-512 Signature Security

Status: **strong**

Verified:

- signing digest and verification use the same network-bound transaction digest path,
- wrong-key/wrong-owner/wrong-network/wrong-signature tests exist in validation,
- block validation parallelizes signature verification but still returns individual failures cleanly.

Remaining gaps:

- no dedicated signature parser fuzz target beyond current tx/witness fuzz coverage,
- no fresh differential harness comparing cacheless vs cached verification behavior across randomized cases.

Falcon Signature Security Grade: **8/10**

8. Phase 5: UTXO Spend Authorization

Status: **stronger and now better defended on restart**

Verified:

- missing, spent, immature, wrong-owner, wrong-network, and legacy-lock UTXO spends are rejected,
- dirty restart now replays canonical blocks and compares persisted UTXOs against rebuilt UTXOs,
- a new property test proves apply-then-disconnect restores the exact UTXO image in memory.

Remaining gaps:

- no persisted state root,
- corrupted non-UTXO datasets need broader recovery coverage.

UTXO Spend Authorization Grade: **8/10**

9. Phase 6: Amount, Fee, Dust, and Overflow Security

Status: **good**

Verified:

- normal transactions cannot output more than inputs,
- fee metadata is no longer trusted as consensus input,
- coinbase is checked against exact subsidy plus recomputed valid fees,
- dust and fee floor rules are exercised in mempool/node tests.

Remaining gaps:

- no randomized miner-vs-validator fee differential suite,
- no large random arithmetic property suite beyond reward additivity.

Amount/Fee/Dust Math Grade: **8/10**

10. Phase 7: Mempool Consensus Alignment

Status: **good**

Verified:

- mempool rejects conflicts, sub-dust entries, and immature spends,
- block validation rechecks transactions instead of trusting mempool admission,
- service tests show invalid raw transaction encodings fail closed before entry.

Remaining gaps:

- no randomized mempool/miner churn differential suite,
- no exhaustive external submission matrix for every policy-vs-consensus edge.

Mempool Consensus Alignment Grade: **8/10**

11. Phase 8: Miner Block Template Security

Status: **good**

Verified:

- mainnet/testnet candidate mining now requires explicit configured payout addresses,
- wrong-network reward addresses are rejected,
- candidate block overflows and invalid mempool entries are handled by tests,
- service exposes canonical header bytes for miners.

Remaining gaps:

- no randomized miner-vs-validator differential harness,
- snapshot bootstrap trust is still operationally weaker than signed-distribution designs.

Miner Block Template Security Grade: **8/10**

12. Phase 9: Coinbase and Monetary Policy Security

Status: **good**

Verified:

- coinbase must be first and unique,
- coinbase witness/tx-PoW shape is stricter,
- reward schedule additivity/monotonicity now has property coverage,
- overclaim and legacy-lock cases are rejected.

Remaining gaps:

- no huge randomized supply-boundary property suite yet,
- no second independent audit of subsidy logic.

Coinbase and Monetary Policy Security Grade: **8/10**

13. Phase 10: Block Validation Security

Status: **good**

Verified:

- invalid blocks are rejected without mutating chainstate,

- tx duplicates and input duplicates are checked,
- PoW, merkle, witness root, parent, target, and timestamp checks are present,
- tx signatures are revalidated inside block validation,
- new restart/replay tests strengthen the persistence side of block acceptance.

Remaining gaps:

- `validate_block_without_pow` should be harder to misuse,
- no full fuzz-runtime proof in this pass.

Block Validation Security Grade: **8/10**

14. Phase 11: Proof-of-Work and Difficulty Security

Status: **good**

Verified:

- validator recomputes PoW from canonical header bytes,
- founder hashes are now committed in headers consistently across codepaths,
- future timestamp ceiling exists,
- work-not-height fork choice tests exist.

Remaining gaps:

- no explicit integrated `hash == target` assertion recorded in this pass,
- local-clock dependence for future-drift checks remains an operational sharp edge.

PoW and Difficulty Security Grade: **8/10**

15. Phase 12: Chain Selection and Reorg Security

Status: **stronger than before**

Verified:

- height-only fork choice bug is tested against work-based selection,

- max reorg boundary tests exist,
- replay-from-genesis property test now exists,
- rollback journal replaced the older full clone-heavy reorg safety path,
- candidate validation failure restoration tests exist.

Remaining gaps:

- crash injection still does not fully cover disconnect/reconnect substeps,
- no large randomized fork-forest differential suite.

Chain Selection and Reorg Security Grade: **8/10**

16. Phase 13: Storage, LMDB, and Crash Safety

Status: **much improved**

Verified:

- commit fault points now cover archive, snapshot, state, and pre-commit windows,
- dirty restart detects UTXO corruption and quarantines state,
- snapshot replacement faults preserve prior committed snapshot and UTXO image,
- cross-network persisted UTXOs fail closed.

Remaining gaps:

- no persisted state root,
- no full fuzz coverage for LMDB value decoders,
- reorg-specific crash windows still need more failpoints.

Storage and Crash Safety Grade: **8/10**

17. Phase 14: Serialization, Binary Codec, and Compatibility Security

Status: **good**

Verified:

- block and tx canonical decoders reject trailing bytes,
- legacy lock formats are rejected,
- mixed network and cross-network replay paths fail closed in storage/validation,
- protocol fixtures assert canonical roundtrip behavior.

Remaining gaps:

- more differential coverage is needed across wallet/node/explorer surfaces,
- fuzz runtime is not yet proven in CI.

Serialization and Codec Security Grade: **8/10**

18. Phase 15: Network Constants and Mainnet Immutability

Status: **good**

Verified:

- per-network genesis/header founders hashes are locked in code,
- wrong-network persisted UTXOs and block wrappers fail closed,
- operator config affects operational settings, not core consensus constants,
- reward-address defaults are dev-only on non-production networks.

Remaining gaps:

- release process should assert `devtools` is absent from production artifacts,
- snapshot bootstrap remains an operator-configured trust edge.

Network Constants and Immutability Grade: **8/10**

19. Phase 16: Legacy, Dead Code, and Bypass Audit

Keyword table

Keyword	File	Function	Reachable ?	Consensus Risk	Severity	Required Action
legacy	crates/atho-storage/src/db.rs	legacy_layout_present	Yes	rejects old on-disk layouts	Low	keep; regression-test
legacy	crates/atho-storage/src/validation.rs	legacy_lock_rejection_paths	Yes	prevents old lock bypass	Low	keep; regression-test
skip_pow	crates/atho-storage/src/validation.rs	validate_block_within_pow	Yes	misuse could under-test consensus	High	reduce visibility / isolate
fallback	GPU miner code	native env helpers	No consensus	operational only	Low	none
trusted	snapshot config/service strings	bootstrap snapshot naming	Yes	operator trust edge	Medium	signed bundles
panic!	crates/atho-node/src/node.rs	load_or_new	Yes	startup availability fail-open confusion	/ High	prefer fallible wrapper
panic!	crates/atho-storage/src/chainstate.rs	load_or_new	Yes	startup availability fail-open	High /	prefer fallible

Keyword	File	Function	Reachable ?	Consensus Risk	Severity	Required Action
				confusion		wrapper
devtools	crates/atho- node/ Cargo.toml	feature gate	Build-time only	misuse in release builds	Medium	release gate
default	many configs	startup defaults	Yes	mostly non- consensus	Low	keep coverage
unwrap/ expect	mostly tests	many test helpers	Mostly no	low in tests, mixed in wrappers	Low/ Medium	isolate remaining production wrappers

Legacy and Bypass Resistance Grade: **7/10**

20. Phase 17: Fuzz Testing Requirements

Existing fuzz targets:

- p2p_frame_decode
- p2p_message_roundtrip
- tx_witness_parse
- tx_decode
- tx_roundtrip
- sighash
- block_decode
- block_template_decode
- block_validate

- mempool_admission
- compact_block_reconstruct
- network_message_decode
- rpc_request_decode
- address_decode

What improved:

- fuzz workspace now builds against current header fields,
- fuzz helpers can still seed chainstate through a gated devtools path rather than an always-on production method.

Still missing:

- LMDB value decoder fuzz target
- undo-data fuzz target
- chainstate loader fuzz target
- explicit public-key parser fuzz target
- CI runtime fuzz execution proof

Fuzz success criteria remain correct:

- no panic,
- no hang,
- no invalid-state mutation,
- no silent acceptance of malformed consensus data.

Fuzz Testing Coverage Grade: **6/10**

21. Phase 18: Property-Based Testing Requirements

Existing property tests now present:

- replay-from-genesis matches live chainstate

- reward schedule stays additive and monotonic
- apply-then-disconnect restores exact UTXO image

Still missing:

- persisted failed-block atomicity property tests
- randomized reorg replay equality over larger histories
- miner-built block always passes validator under random mempool sets
- wrong-network always rejects across random object generators
- signature-always-rejects-invalid property suite

Property Test Coverage Grade: **7/10**

22. Phase 19: Differential Testing Requirements

Still largely missing and important:

- wallet txid vs node txid
- wallet serialization vs node parsing
- miner block vs validator block acceptance
- reindex state vs live state across randomized histories
- explorer balance vs canonical UTXO scan
- signature cached vs fresh verification parity

Differential Testing Grade: **5/10**

23. Phase 20: Regression Test Requirements

Bug	Severity	Test Name	File	Fails	Passes	CI
				Before	After	Required
				Fix?	Fix?	?
tx-PoW witness/iden	Critical	existing validation	tx-PoW crates/atho-p2p, regressions	Yes	Yes	Yes

				Fails	Passes	CI
Bug	Severity	Test Name	File	Before	After	Required
				Fix?	Fix?	?
tity ambiguity		plus compact block reconstruction tests	crates/atho-storage			
fee metadata trusted as consensus input	Critical	fee metadata/coinbase exact-fee regressions	crates/atho- storage/src/ validation.rs	Yes	Yes	Yes
dirty restart trusted persisted UTXOs	Critical	dirty_restart_d etects_persiste d_utxo_corrupti on_and_quaranti nes_state	crates/atho- storage/src/ chainstate.rs	Yes	Yes	Yes
snapshot replacement could tear committed state	High	full_snapshot_c ommit_faults_le ave_prior_snaps hot_intact	crates/atho- storage/src/db.rs	Yes	Yes	Yes
commit fault handling too narrow	High	commit_fault_in jection_rolls_b ack_chainstate_ mutation	crates/atho- storage/src/ chainstate.rs	Yes	Yes	Yes
replay proof missing	High	replay_from_gen esis_matches_li ve_chainstate	crates/atho- storage/src/ chainstate.rs	N/A	Yes	Yes

Bug	Severity	Test Name	File	Fails Before Fix?	Passes After Fix?	CI Required ?
reward additivity proof missing	High	reward_schedule _stays_additive _and_monotonic	crates/atho- storage/src/ chainstate.rs	N/A	Yes	Yes
UTXO apply/discon nect rollback missing	exact High	apply_then_disc onnect_restores _exact_utxo_image	crates/atho- storage/src/utxo.rs	N/A	Yes	Yes
stale adversarial lock format	Medium	refreshed canonical lock helpers	crates/atho-node/ src/bin/atho- adversarial.rs	Yes	Yes	Yes
fuzz workspace compile break	Medium	cargo check -- manifest-path fuzz/Cargo.toml --all-targets	workspace gate	Yes	Yes	Yes

Regression Test Quality Grade: **8/10**

24. Severity Rating System

- **CRITICAL:** inflation, double spend, invalid block acceptance, valid block rejection, chain split, signature bypass, or consensus-state corruption
- **HIGH:** node crash, DoS, reorg corruption, wrong balances, storage inconsistency, dangerous production behavior

- **MEDIUM:** degraded validation, missing proof coverage, risky assumptions, incomplete edge testing
- **LOW:** code quality or operational polish without current consensus impact

Key findings in this pass

Severity	File	Function/ Class	Description	Exploit/ failure scenario	Consensus impact	Fix	Team
High	crates/atho-storage/src/validation.rs	validate_block_without_pow	public no-custom tooling/tests may validate still wrong path helper exists	under-tested acceptance path	scope make test/dev-only	or make re-test	
High	crates/atho-node/src/node.rs	load_or_new	panic wrapper snapshot bootstrap failure	corrupted/misconfigured snapshot causes abort not failure	operational safety restart typed predictability	prefer / fallible production wrapper	stable failure
High	crates/atho-storage/src/chainstate.rs	load_or_new	panic wrapper state failure	local corruption aborts instead of structured fail-closed	operational safety	prefer fallible wrapper	stable failure
High	crates/atho-node/src/node.rs	snapshot bootstrap	hash-pinned but snapshot	operator pins wrong snapshot or	wrong-state risk outside	signed snapshot	signature sn

Severity	File	Function/ Class	Description	Exploit/ failure scenario	Consensus impact	Fix	Test re
			trust	compromised distribution	chain rules	bundles	test
High	fuzz/	workspace/ runtime coverage	fuzz compile fixed but runtime not proven in CI	parser/storage bugs survive latent because fuzz consensus/pa never actually rser risk runs		CI fuzz execution gate	fu CI
Medium	crates/atho- storage/src/db.rs	recovery verification	no persisted state root	startup integrity requires full replay scan	slower/less direct integrity proof	add state sta root or ve equivalent	test
Medium	crates/atho- storage/src/db.rs	failpoints	reorg disconnect/re connect windows not fully fault- injected	mid-reorg crash not exhaustively simulated hide	possible dirty-state bug could hide	add more re failpoints test	test
Medium	crates/atho- storage/src/ validation.rs	future timestamp ceiling	depends on local wall clock	badly skewed honest nodes may differ operationally	temporary divergence risk	document clock discipline or bounded MTP policy	sk bo test

Severity	File	Function/ Class	Description	Exploit/ failure scenario	Consensus impact	Fix	T re
						refinement	
Medium	crates/atho-node/ src/bin/atho- adversarial.rs	many cases	harness still false comfort leans on no- about full PoW block acceptance proof gap	validator path		add full- ac path cases C	
Medium	crates/atho-wallet + node/core	serializer parity	no valid wallet tx wallet/node could drift reject-valid / differential from node txid drift risk suite interpretation			add differential suite	w di

25. Full Subsystem Grading Table

Subsystem	Grade	Mainnet Ready?	Biggest Vulnerability	Missing Tests	Required Fix
Network/API input security	8/10 Almost		incomplete malformed raw ingress matrix	wrong-network submissions	raw expand ingress tests
Transaction parsing	8/10 Almost		no long-run parser fuzz proof	parser fuzz runtime	run fuzz in CI
Transaction hashing	8/10 Almost		missing wallet/node txid differential	txid diff suite	add differential tests
Falcon signature	8/10 Almost		limited dedicated sig/pubkey fuzz		add dedicated

Subsystem	Grade	Mainnet Ready?	Biggest Vulnerability	Missing Tests	Required Fix
security			signature fuzzing		fuzz target
UTXO spend authorization	8/10 Almost		no persisted state root	corrupted-record matrix	add state root / more corruption tests
Fee/dust/amount math	8/10 Almost		no randomized arithmetic miner/validator diff suite	diff/property	add differential tests
Mempool alignment	8/10 Almost		no randomized mempool/miner churn differential tests	churn	add differential tests
Miner template security	8/10 Almost		no randomized full candidate-vs-validator differential suite		add differential tests
Coinbase validation	8/10 Almost		no second more independent review properties	boundary	re-audit subsidy/coinbase edge cases
Monetary policy	8/10 Almost		supply proofs still modest	more boundary tests	broaden property coverage
Block validation	8/10 Almost		public no-PoW full-path helper tests	adversarial	reduce helper scope
PoW/difficulty	8/10 Almost		missing equality boundary in this pass	exact hash == target integration test	add boundary tests

Subsystem	Grade	Mainnet Ready?	Biggest Vulnerability	Missing Tests	Required Fix
Chain selection/reorg	8/10	Almost	incomplete reorg disconnect/reconnect crash injection	fault tests	add failpoints/tests
Storage/crash safety	8/10	Almost	no persisted state root, incomplete LMDB/undo fuzz dataset fuzzing		add fuzz + state root
Serialization/codec	8/10	Almost	differential thin	testing wallet/node/explorer diff tests	add differential tests
Network constants	8/10	Almost	release artifact no-devtools feature discipline	assertion	release add gate
Legacy bypass resistance	7/10	Not yet	validate_bloc k_without_pow, bypass misuse tests panic wrappers		narrow helper scope
Fuzz testing	6/10	Not yet	runtime fuzz not proven	all fuzz runtime lanes	enforce fuzz CI
Property testing	7/10	Not yet	coverage narrow	still more invariants	expand proptests
Differential testing	5/10	Not yet	mostly absent	wallet/miner/reindex diffs	build diff harnesses
Regression testing	8/10	Almost	some new risks still lack regressions	ingress/crash/signature gaps	add targeted regressions
CI mainnet	6/10	Not yet	no combined mainnet gate lane		add mandatory

Subsystem	Grade	Mainnet Ready?	Biggest Vulnerability	Missing Tests	Required Fix
gate			fuzz/property/diff/crash gate		gate
Overall			proof net still		
consensus	7/10	Not yet	thinner than fuzz/diff/crash gaps		finish mainnet gate work
security			validator quality		

26. Mainnet Blocking Conditions

Current assessment against the explicit block list:

- Invalid block can be accepted: **not reproduced**
- Valid block can be rejected due to nondeterminism: **not reproduced**
- Inflation is possible: **not reproduced**
- Coinbase overclaim is possible: **not reproduced**
- Double spend is possible: **not reproduced**
- Immature coinbase spend is possible: **not reproduced**
- Signature verification can be bypassed: **not reproduced**
- UTXO state can mutate after failed validation: **not reproduced**
- Reorg can corrupt balances: **not reproduced**, but crash-window proof incomplete
- Serialization can produce different hashes across nodes: **not reproduced**
- Mainnet constants can be changed at runtime: **not reproduced**
- Miner and validator disagree: **not reproduced**, but differential proof thin
- Mempool and block validation disagree on consensus rules: **not reproduced**, but broader matrices still needed
- Legacy code can bypass current rules: **not reproduced**
- Storage corruption can be silently trusted: **substantially improved**, but broader dataset coverage still needed
- Failed block can update chain tip: **not reproduced**

- Failed block can write UTXOs: **not reproduced**
- API/RPC can bypass validation: **not reproduced**
- Testnet/regtest data can enter mainnet: **not reproduced**
- PoW target comparison is inconsistent: **not reproduced**
- Chain selection uses height instead of cumulative work: **not reproduced**
- Reindex from genesis does not match live state: **not reproduced** in current replay property scope
- Coinbase maturity is not enforced in block validation: **not reproduced**
- Fee calculation differs between miner and validator: **not reproduced**
- Falcon signature message differs between signing and verifying: **not reproduced**
- There is no regression test for a fixed CRITICAL bug: **false**; several now exist

Decision implication:

- I do **not** see a live reproduced critical consensus bug in the current code.
- I still recommend delaying mainnet because the remaining fuzz/differential/crash proof net is thinner than I want for a chain claiming production hardness.

27. Final Mainnet Gate Checklist

Before mainnet, all of the following should be green:

- ☒ consensus unit tests present and substantial
- ☒ block validation adversarial tests present
- ☒ Falcon signature negative tests present
- ☒ coinbase and monetary boundary tests present
- ☒ UTXO atomicity regressions present
- ☒ serialization canonical tests present
- ☒ work-not-height reorg tests present
- ☒ startup recovery UTXO replay verification present
- ☐ fuzz targets executed in CI for minimum duration
- ☐ LMDB/undo/chainstate loader fuzz targets added

- [] randomized miner-vs-validator differential tests
- [] randomized wallet-vs-node differential tests
- [] reindex/live-state randomized differential tests
- [] reorg disconnect/reconnect crash failpoints
- [] signed snapshot verification if bootstrap snapshots remain enabled
- [] release artifact check forbidding `devtools`
- [] combined mainnet gate CI lane running fuzz + property + differential + crash suites

28. Final Deliverables

This report provides:

1. `ATHO_CONSENSUS_SECURITY_VULNERABILITY_AND_PHASE_TEST_AUDIT.md`
2. a phase-by-phase consensus map
3. a threat model
4. a vulnerability list
5. an edge-case and missing-test matrix
6. adversarial block/transaction concerns
7. Falcon signature attack/test requirements
8. UTXO atomicity coverage summary
9. monetary policy boundary coverage summary
10. coinbase overclaim coverage summary
11. fee/dust/overflow coverage summary
12. serialization/fuzz plan
13. PoW/difficulty coverage summary
14. reorg/fork test requirements
15. storage crash recovery test requirements
16. API/RPC bypass test requirements
17. legacy/dead-code bypass report
18. property-testing plan

19.differential-testing plan

20.regression-test list

21.subsystem grade table

22.mainnet launch decision

23.required-fix list

24.missing-test list

Final Verdict

Atho is no longer in the same shape as the earlier blocked audit state.

The recent fixes closed real issues:

- fuzz build break fixed,
- stale adversarial lock assumptions fixed,
- property testing added,
- persisted UTXO integrity replay added,
- storage failpoint coverage widened,
- dev-only chainstate seeding gated.

That moves Atho from "too many unresolved proof gaps" to "plausibly launchable after one more hardening pass."

My decision remains:

MAINNET DELAY RECOMMENDED

Not because I found a fresh smoking-gun inflation or signature bypass bug in the current path, but because the remaining fuzz, differential, release-gating, and reorg-crash proof gaps are exactly the sort of things that turn into ugly surprises after launch.

Appendix D: Full Vulnerability Table

Appendix D.1. Summary vulnerability inventory

Severity	Area	Example	Why It Matters	Current Status
Critical	Block validation	Invalid block accepted	Consensus failure	No live example reproduced in latest local audit
Critical	Monetary policy	Coinbase overclaim	Inflation	Strong validator checks present
Critical	UTXO accounting	Failed validation mutates state	Silent divergence	Improved, but always test further
High	Storage	Corrupt persisted UTXO image trusted	Wrong balances / accept/reject outcomes	Dirty-start replay checks improved
High	Differential coverage	Wallet/miner/node drift	Hidden consensus mismatch	Still light
High	Fuzz runtime	Parser survives	panic DoS and safety gap	Build checks present; runtime needs more CI weight
High	Reorg crash coverage	Disconnect/reconnect boundary bug	Balance corruption	Better than before, still incomplete
Medium	Snapshot trust	Hash pin only	Operational trust	Future signed-distribution

Severity	Area	Example	Why It Matters	Current Status
High			weakness	hardening recommended

Appendix D.2. Incorporated internal full consensus audit extract

Atho Full Consensus Audit and Production Grade

1. Executive Summary

Overall Verdict

- Overall production readiness grade: **4/10**
- Overall consensus safety grade: **4/10**
- Mainnet launch decision: **MAINNET BLOCKED**
- External audit recommended: **Yes**

Why launch is blocked

The codebase has a strong amount of localized validation logic and much better reorg handling than a toy chain, but I cannot call it production-safe because several consensus-critical rules are either under-committed or rely on state that is not fully verified at restart.

The two biggest blockers are:

1. **Consensus-relevant block data is not fully committed by the block header.**
 - `tx_pow_nonce` / `tx_pow_bits` are validated during block acceptance, but they are **not** committed by either the txid merkle root or the witness root.
 - `fees_total_atoms` / `fees_miner_atoms` are used during block validation, but they are **not** part of the canonical block bytes used by block-file storage, and they are not committed by the header.
 - Files: [crates/atho-core/src/transaction.rs](#), [crates/atho-core/src/block.rs](#), [crates/atho-storage/src/validation.rs](#)

2. **The miner reward output is deterministically derived from public data.**
 - `reward_target_for_height()` derives a Falcon keypair from `sha3_384(network.domain_tag || height)`, which means **anyone can derive the private key and steal the reward** for that height once mature.
 - File: [crates/ath0-node/src/mining.rs](https://github.com/ath0/ath0-node/blob/master/src/mining.rs)

Top 10 Highest-Risk Findings

1. **Critical:** block header does not commit `tx_pow_nonce` / `tx_pow_bits`, but validation depends on them.
2. **Critical:** mining rewards are sent to deterministic public keys derived from height and network.
3. **Critical:** startup consistency checks do not verify that persisted LMDB UTXOs match canonical block history.
4. **High:** coinbase witness bytes are not explicitly constrained, and malformed witness bytes collapse to the same witness commitment as empty witness bytes.
5. **High:** `Transaction::from_full_bytes()` accepts truncated full-transaction encodings with missing tx-PoW fields and defaults them to zero.
6. **High:** canonical block/transaction decoders allocate from attacker-controlled counts before enforcing sane bounds.
7. **High:** there is no upper bound on future block timestamps; only the lower MTP rule is enforced.
8. **High:** compact-block reconstruction uses txid-based short IDs even though tx-PoW fields are not committed by txid.
9. **Medium:** consensus-vs-policy comments are misleading in places where dust and fee floors are actually enforced during block validation.
10. **Medium:** miner template construction does not self-validate the completed candidate before exposing it to miners or RPC clients.

Top 10 Fixes Required Before Mainnet

1. Commit **all** consensus-relevant transaction bytes into the block header commitment path.

2. Replace deterministic mining reward keys with operator-configured or wallet-controlled reward destinations.
3. Add a persisted UTXO integrity root or deterministic startup rebuild/verification path.
4. Make coinbase witness and tx-PoW fields strict: empty witness only, zero tx-PoW only.
5. Make `Transaction::from_full_bytes()` strict: missing tx-PoW tail must be rejected.
6. Bound all pre-allocation counts in manual decoders before `Vec::with_capacity(...)`.
7. Add a deterministic future timestamp ceiling.
8. Remove `fees_total_atoms / fees_miner_atoms` from consensus validity, or serialize and commit them canonically.
9. Make compact-block reconstruction use a commitment that includes the same bytes block validity depends on.
10. Add missing regression tests listed in Section 17.

Top 10 Tests That Must Exist Before Mainnet

1. Block with altered tx-PoW fields but unchanged header commitments must be rejected or impossible to construct.
2. Block with altered `fees_total_atoms / fees_miner_atoms` and unchanged header must not create an alternate validity outcome.
3. Future-dated block above allowed drift must be rejected.
4. Coinbase with non-empty witness must be rejected.
5. Coinbase with non-zero `tx_pow_nonce / tx_pow_bits` must be rejected.
6. Full transaction encoding missing tx-PoW tail must be rejected.
7. Corrupted persisted UTXO set on dirty restart must be detected and repaired or fail closed.
8. Compact block reconstruction with same `txid` but different tx-PoW bytes must not silently accept the wrong variant.
9. Candidate block produced by the miner must round-trip through block validation

successfully.

10. Two-coinbase and coinbase-not-first blocks must be explicitly tested.

2. Consensus-Critical System Map

Area	File	Function/Class	Consensus Role	Risk Level
Block header hashing	crates/athocore/src/block.rs	BlockHeader::canonical_bytes, BlockHeader::block_hash	Defines block identity and PoW hash input	Critical
Block body commitments	crates/athocore/src/block.rs	merkle_root, witness_root, Block::from_canonical_bytes	Binds transactions to headers	Critical
Transaction serialization	crates/athocore/src/transaction.rs	base_bytes, full_bytes, from_full_bytes, txid, wtxid, witness_commitment_hash	Defines txids, witness commitments, storage decode	Critical
Signature digest rules	crates/athocore/src/consensus/signatures.rs	transaction_signature_digest, transaction_signature_digest_for_input_indexes	Defines the exact Falcon message being signed	Critical
Difficulty/	crates/athocore/src/consensus/signatures.rs	target_for_next_block	PoW target	Critical

Area	File	Function/Class	Consensus Role	Risk Level
chainwork	core/src/consensus/pow.rs	ock_with_timestamp, meets_target, compare_branch_work	validation and branch selection	
Emission schedule	crates/athores/core/src/consensus/subsidy.rs	block_subsidy_atoms_for_network, cumulative_issued_*	Subsidy and issuance schedule	High
Consensus versions	crates/athores/core/src/consensus/rules.rs	rules_at_height, is_supported_version	Height-gated ruleset/version enforcement	High
Network separation	crates/athores/core/src/network.rs	consensus_id, p2p_magic, visible_prefix, utxo_flag	Prevents cross- network replay and storage mixing	High
Genesis anchoring	crates/athores/core/src/genesis.rs	genesis_state, genesis_hash	Anchors network identity and initial UTXO state	Critical
Address ownership	crates/athores/core/src/address.rs	public_key_digest, payment_digest_from_locking_script	Canonical 32-byte ownership lock rules	Critical
Fee/dust/tx-	crates/athores-	minimum_required_f	Current relay and	Critical

Area	File	Function/Class	Consensus Role	Risk Level
PoW	core/src/consensus/tx_policy.rs	ee_atoms, required_tx_pow_bits, transaction_pow_is_valid_for_bits	block acceptance transaction rules	
Transaction validation	crates/athos-storage/src/validation.rs	prepare_transaction_validation, validate_transaction_with_context_*	Structural, contextual, ownership, maturity, checks fee, tx-PoW	Critical
Block validation	crates/athos-storage/src/validation.rs	validate_block_with_context_and_schedule, validate_contextual_header_precheck	Full block acceptance path	Critical
UTXO state machine	crates/athos-storage/src/utxo.rs	UtxoSet::apply_block, disconnect_block, is_spendable_at	UTXO updates, maturity, rollback	Critical
Chainstate reorgs	/crates/athos-storage/src/chainstate.rs	connect_block, select_branch, switch_branch_incrementally, replace_with_valid	Canonical selection and recovery chain reorg	Critical

Area	File	Function/Class	Consensus Role	Risk Level
		ated_branch		
LMDB + flat files	crates/atho-storage/src/db.rs	commit_chainstate, replace_chainstate, run_startup_consistency_checks, apply_utxo_delta	Persistence of snapshots, UTXOs, Critical blocks, txs	
Data-dir separation	crates/atho-storage/src/path.rs	database_dir, block_storage_dir, rpc_cookie_path	Network-isolated storage paths	High
Miner template assembly	crates/atho-node/src/mining.rs	build_candidate_block, reward_target_for_height	Assembles blocks miners actually solve	Critical
Node block acceptance	crates/atho-node/src/node.rs	connect_block, submit_block, consider_branch	Consensus entrypoints from runtime and P2P	Critical
Mempool admission	crates/atho-node/src/mempool.rs	admit, revalidate, reserve_inputs	Policy admission and double-spend blocking	High
RPC transaction admission	crates/atho-node/src/service.rs	parse_raw_transaction_hex, broadcast_transaction	Externally exposed transaction submission path	High

Area	File	Function/Class	Consensus Role	Risk Level
		ion_value		
P2P transport	crates/athorp2p/src/protocol.rs	MessagePayload::Block, compact_block_from_block, reconstruct_compact_block	Wire serialization and compact block reconstruction	Critical
Raw storage	crates/athor-storage/src/block_files.rs	canonical_block_payload storage	Persists block bytes for restart/recovery	High

Consensus-Critical Storage Paths

Storage Path / Dataset	Purpose	Risk Level
LMDB meta	chain snapshot, schema version, storage metadata, runtime state	Critical
LMDB blocks	per-block metadata and chainwork flags	Critical
LMDB block_heights	canonical height -> hash map	Critical
LMDB block_transactions	block -> ordered txid list	High
LMDB transactions	tx archive records	High
LMDB utxos	live UTXO set	Critical

Storage Path / Dataset	Purpose	Risk Level
blocks/*.dat flat files	canonical block payload archive	Critical
chainstate.commit-journal	crash recovery marker	High

3. Block Validation Audit

Block Validation Rule Matrix

Rule	Where Enforced	Complete ?	Tested?	Failure Mode	Fix Needed
Non-empty block	validation.rs: 839-841	Yes	Yes	Empty block accepted if bypassed elsewhere	Keep
Block version at height	validation.rs: 842-844	Yes	Yes (future_block_version_i s_rejected_before_activation)	Chain split on version drift	Keep
Network id match	validation.rs: 845-847	Yes	Partial	Cross-network block acceptance	Add explicit mainnet/testnet mismatch test
Height match	validation.rs: 848-850	Yes	Partial	Valid block rejected if invalid height	Add explicit / bad-height height

Rule	Where Enforced	Complete ?	Tested?	Failure Mode	Fix Needed
				accepted	block test
Timestamp nonzero	validation.rs: 851-853	Yes	Partial	Zero timestamp block accepted	Add explicit test
Lower timestamp bound (MTP+1)	validation.rs: 1220-1224, pow.rs:387-389	Yes	Yes	Time-travel below MTP	Keep
Upper future timestamp bound	Missing	No	No	Future-dated blocks distort chain deterministic timing/difficult y	Add future cap
Target bounds	validation.rs: 865-868	Yes	Yes	Out-of-range target accepted	Keep
Exact expected target	validation.rs: 1225-1227	Yes	Yes (contextual_ validation_ rejects_unex pected_target)	Invalid retarget accepted	Keep
PoW hash <= target	validation.rs: 1228-1230, 954-	Yes	Yes	Invalid header	Keep

Rule	Where Enforced	Complete?	Tested?	Failure Mode	Fix Needed
	956			accepted	
Merkle root	validation.rs: 872-875	Yes	Partial	Body/header mismatch	Add explicit bad-merkle test
Witness root	validation.rs: 876-879	Partial	Yes (header_witness_root_must_match_body_commitment)	Malformed witness bytes can collapse to empty same commitment	Commit raw witness bytes or reject non-malformed witness in all paths
Coinbase first	collect_prepared... validates tx[0] as coinbase	Partial	Partial	Non-first coinbase rejection implicit, explicit	Add is test not
Exactly one coinbase	Remaining txs rejected via prepare_transaction_validation on empty inputs	Partial	No explicit test	Second coinbase rejected, through generic path	Add explicit but two-coinbase test
Duplicate txid rejection	validation.rs: 910-912	Yes	Partial	Same txid twice in block	Add one duplicate-txid block test

Rule	Where Enforced	Complete ?	Tested?	Failure Mode	Fix Needed
Duplicate input rejection across block	validation.rs: 913-915, 1163- 1168	Yes	Partial	Intra-block double spend	Add direct duplicate-input block test
Block size / weight / vsize	validation.rs: 822-830	Yes	Yes	Oversized block accepted	Keep
Fee sum exact	validation.rs: 1196-1200	Partial risky	Partial	Depends on uncommitted fee metadata	Remove fee metadata from consensus, compute fee exactness directly
Coinbase reward exact	validation.rs: 900-909, 796- 819	Partial	Partial	Depends on coinbase fields plus fee metadata	Compare coinbase output directly to subsidy + computed fees
UTXO transition only after full	chainstate.rs: 248-281	Yes	Yes	Dirty state after failed block	Keep

Rule	Where Enforced	Complete ?	Tested?	Failure Mode	Fix Needed
------	----------------	---------------	---------	--------------	------------

validation

Block Validation Findings

Strengths

- Block validation is centralized instead of being spread across wallet, miner, mempool, and storage paths.
- Contextual validation checks the parent hash, expected target, PoW, duplicate block inputs, fee exactness, and maturity before mutating state.
- Reorg-specific validation reuses the same block validation logic through a UTXO overlay rather than trusting mempool prevalidation.

Critical Gaps

1. **Consensus-critical block validity depends on uncommitted fields.**
 - `tx_pow_nonce` / `tx_pow_bits` are checked during transaction validation inside block validation.
 - But the header only commits `txid` merkle root and `witness_commitment_hash`, and the current witness commitment hash excludes tx-PoW fields.
 - Files: `transaction.rs:616-623`, `block.rs:438-477`, `validation.rs:628-633, 661-666`
2. **Fee metadata is used as if it were consensus input.**
 - `validate_block_with_context_and_schedule()` rejects blocks when `sum_fees != block.fees_total_atoms` or `fees_total_atoms != fees_miner_atoms`.
 - Those fields are not in `Block::canonical_bytes()` and are not committed by the header.
 - Files: `block.rs:269-297`, `validation.rs:1196-1200`

3. **Future timestamp bound is missing.**

- Current logic only enforces $\text{timestamp} \geq \text{MTP} + 1$.
- A miner can stamp blocks arbitrarily far into the future and still satisfy validation.

4. **Coinbase witness handling is too loose.**

- Coinbase validation does not require empty witness or zero tx-PoW fields.
- Malformed witness bytes are effectively omitted from the witness commitment hash.

Block Validation Grade: 5/10

Why:

- The validator checks many important rules correctly.
- But a block must be a self-authenticating consensus object. Right now, the header does not commit all of the data that block acceptance depends on. That is a launch blocker.

4. Transaction Validation Audit

Transaction Rule Matrix

Rule	Where Enforced	Complete ?	Tested ?	Failure Mode	Fix Needed
Supported tx version	<code>validation.rs:354-358</code>	Yes	Yes	Version drift	Keep
Non-coinbase txs must have inputs	<code>prepare_transaction_validation()</code>	Yes	Partial	Empty-input normal accepted	tx Add explicit test
Outputs must exist	<code>validation.rs:359-361</code>	Yes	Partial	Burn-only malformed accepted	or tx Add explicit test
Output count	<code>validation.rs:3</code>	Yes	Yes	DoS/huge	Keep

Rule	Where Enforced	Complete ?	Tested ?	Failure Mode	Fix Needed
cap	62-364			fanout	
Raw/vsize caps	validation.rs:3 65-369	Yes	Yes	Oversized tx accepted	Keep
No zero outputs	validation.rs:3 70-372	Yes	Partial	Value ambiguity weird accounting	/ Add explicit test
Dust floor	validation.rs:3 83-390	Yes	Yes	Spam / block-policy drift	Keep, but clarify consensus-vs-policy docs
Duplicate input rejection	validation.rs:3 73-378	Yes	Partial	Double spend within tx	Add explicit direct test
Fee floor	validation.rs:3 79-381, 658-659	Yes	Yes	Fee-below-floor tx accepted	Keep
Canonical 32-byte output locks only	validation.rs:3 91-393, 463-468	Yes	Yes	Legacy lock accepted	Keep
Witness present parseable	validation.rs:3 94-399	Yes	Yes	Malformed witness bypass	Keep
Signer-group coverage of all	validation.rs:4	Yes	Partial	Uncovered input	Add targeted test or

Rule	Where Enforced	Complete ?	Tested ?	Failure Mode	Fix Needed
inputs	00-455			duplicate ref	
Falcon pubkey/signature lengths	validation.rs:2 77-282	Yes	Yes	Truncated/ oversized witness accepted	Keep
Falcon signature digest	validation.rs:2 87-297, signatures.rs:6 0-81	Yes	Yes	Wrong-message signature accepted	Keep
Ownership digest binding	validation.rs:6 88-700	Yes	Yes	Wrong pubkey spends UTXO	Keep
Network/ genesis replay protection	signatures.rs:6 0-81, tx_policy.rs:18 6-193	Yes	Yes	Cross-network replay	Keep
Maturity confirmations	validation.rs:7 / 01-703, utxo.rs:71-92	Yes	Partial	Immature spend accepted	Add explicit coinbase maturity test at validator level
Exact arithmetic	fee validation.rs:7 04-717, 605-636	Yes	Yes	Inflation/ underflow	Keep
Tx-PoW bits/nonce	validation.rs:5 45-552, 628-633,	Yes	Yes	Spam bypass	rule Commit fields into block

Rule	Where Enforced	Complete ?	Tested ?	Failure Mode	Fix Needed
	661-666				commitment path
Canonical full decode strictness	transaction.rs: 668-770	No	No	Truncated full encoding accepted	Make missing tx- PoW tail invalid
Allocation bounds before decode	transaction.rs: 719-746, TxWitness::from _bytes	No	No	OOM / parser DoS	Bound counts before allocation

Transaction Validation Findings

Strengths

- Ownership validation is good: the unlocking script must match the canonical 32-byte lock, and the signer pubkey must hash to the same digest.
- Network and genesis hash are mixed into the Falcon signing digest, which is the right move for replay separation.
- Exact fee computation uses checked arithmetic and rejects overspends cleanly.
- Legacy non-32-byte locking scripts are explicitly rejected.

High-Risk Gaps

1. **The full transaction decoder is not strict.**
 - Transaction::from_full_bytes() defaults missing tx-PoW tail bytes to (0, 0) if the payload ends after lock_time.
 - File: transaction.rs:750-760
2. **Parser allocations are attacker-controlled before caps are enforced.**
 - Vec::with_capacity(input_count), Vec::with_capacity(output_count), and witness group allocations happen

before any tight sanity bound derived from remaining bytes.

3. Tx-PoW is consensus-relevant but not committed into the header.

- This is partly a serialization issue, partly a transaction-validity issue.

Transaction Validation Grade: 7/10

Why:

- The live validation logic for amounts, ownership, signatures, locks, and fee math is reasonably strong.
- The main downgrades are serialization strictness and the fact that tx-PoW bytes are required for validity but are not committed by the header.

5. UTXO Accounting Audit

UTXO Audit Summary

Check	Status	Notes
UTXO creation after valid block	Good	<code>UtxoSet::apply_block, create_outputs</code>
UTXO spend/delete	Good	<code>spend_inputs, UtxoSet::remove</code>
Duplicate UTXO prevention	Good	<code>UtxoSet::insert, db.rs:1446-1448</code>
Atomic DB updates	Good	<code>commit_chainstate()</code> LMDB rw txn + block archive append journaling
Failed block leaves dirty state	Good memory	in <code>apply_block()</code> rolls back via <code>disconnect_block(undo)</code>
Reorg rollback	Good	Incremental journal + full validated rewrite fallback

Check	Status	Notes
Startup persisted state verification	Weak	Snapshot/tip checked, UTXO contents not checked
Dirty restart false-validity risk	High	Corrupted UTXOs can survive restart
Missing spent-key handling	Weak	<code>apply_utxo_delta()</code> treats missing key delete as idempotent

Deep Findings

- In-memory UTXO mutation path is much better than the persisted restart path.**
 - During live operation, validation happens before `apply_block`, and `apply_block()` can roll itself back.
 - Files: `chainstate.rs:248-281`, `utxo.rs:149-190`
- Dirty restart trust model is not sufficient for production.**
 - `run_startup_consistency_checks()` only proves that the snapshot tip matches block metadata.
 - It does not prove the persisted UTXO set matches the canonical chain.
 - Files: `db.rs:1006-1063`, `chainstate.rs:1400-1411`
- LMDB divergence can be masked.**
 - `apply_utxo_delta()` accepts `LmdbError::NotFound` on spent-key deletes as an idempotent delete.
 - That is practical for dev/recovery scenarios, but unsafe as a permanent production default because it can hide prior UTXO corruption.
 - File: `db.rs:1421-1430`

UTXO Accounting Grade: 5/10

Why:

- The live UTXO state machine is decent.
- The restart/recovery trust model is not strong enough for production because persisted UTXO correctness is not fully re-proven after an unclean shutdown or storage tampering event.

6. Monetary Policy Audit

Monetary Policy Findings

- Reward schedule is deterministic and centralized in `subsidy.rs`.
- Checked arithmetic is used for coinbase reward calculation and fee addition.
- Cumulative issuance helpers exist and are tested at schedule boundaries.
- There is **no finite max supply cap** in the current code by design.
- `max_supply_atoms_for_network()` returns `None`.
- Files: `subsidy.rs:113-118`, `params.rs:20-21`, `40`

Monetary Policy Risk Notes

Check	Status	Notes
Block subsidy calculation	Good	<code>subsidy.rs:38-65</code>
Halving/tail emission	Good	Tail emission floor is explicit
Max supply enforcement	N/A / design-specific	No hard cap exists
Fee inclusion in reward	Good	Coinbase reward checked against subsidy + fees
Overflow risks	Good	<code>checked_add</code> , <code>checked_sub</code> , <code>checked_output_value_atoms</code>

Check	Status	Notes
Mainnet/testnet/regtest separation	Neutral	Same schedule across all networks right now
Miner overclaim prevention	Good if fee metadata issue fixed	Overclaim blocked by coinbase amount + fee exactness

Monetary Policy Grade: 8/10

Why:

- The issuance math itself is clear and tested.
- This score is not 10/10 because block fee metadata is currently part of the validation decision in an under-committed way, which bleeds into coinbase economics.

7. Coinbase Validation Audit

Findings

Rule	Status	Notes
Exactly one coinbase	Partial	Implicitly enforced because only <code>transactions[0]</code> may be coinbase and remaining txs must have inputs
Coinbase first	Partial	Same as above; needs explicit test
Coinbase output count == 1	Good	<code>validation.rs:809-811</code>
Coinbase reward exact	Good but coupled to fee metadata	<code>validation.rs:813-818, 900-909</code>
Coinbase maturity	Good	<code>utxo.rs:77-92</code>
Coinbase	Good	Standard transaction base serialization

Rule	Status	Notes
deterministic txid		
Coinbase witness restrictions	Missing	No explicit <code>witness.is_empty()</code> rule
Coinbase tx-PoW restrictions	Missing	No explicit <code>tx_pow_nonce == 0 && tx_pow_bits == 0</code> rule

Critical Gap

Because the coinbase witness and tx-PoW fields are not explicitly constrained, and malformed non-empty witness bytes can collapse to the same witness commitment as empty bytes, the coinbase path is under-specified.

Coinbase Validation Grade: 4/10

Why:

- Reward amount and basic shape are checked.
- The missing coinbase witness/tx-PoW strictness is too large a consensus ambiguity to ignore.

8. Falcon-512 Signature Validation Audit

Findings

Check	Status	Notes
Correct message signed	Good	<code>signatures.rs:60-81</code>
Correct message verified	Good	<code>validation.rs:287-297</code>
Network binding	Good	<code>network.consensus_id()</code> included
Genesis binding	Good	<code>genesis_hash(network)</code> included

Check	Status	Notes
Truncated/oversized signature rejected	Good	Exact length checks in witness parser and verifier
Wrong public key rejected	Good	Digest must match canonical UTXO lock
Wrong address rejected	Good	Lock digest match enforced
Replay across networks	Good	Explicit tests exist
Batch verification safety	Good enough	No shared batch cache; parallel verification is independent
Signature cache correctness	N/A	No cache used

Falcon Signature Validation Grade: 8/10

Why:

- The signature-domain separation and network/genesis binding are strong.
- The main reason this is not higher is that block-level commitment gaps around tx-PoW/witness bytes can still make block payload identity ambiguous even when the Falcon verifier itself is correct.

9. Serialization / Hashing Audit

Critical Serialization Findings

1. **Block header does not commit all consensus-relevant transaction bytes.**
 - txid() uses base bytes only: transaction.rs:600-608
 - witness_commitment_hash() uses base bytes + parsed witness commitment only: transaction.rs:616-623

- `witness_root()` commits only `witness_commitment_hash()`:
`block.rs:457-477`
 - `tx_pow_nonce` and `tx_pow_bits` are therefore validated but not committed by the header.
2. **Block fee metadata is not in canonical block bytes.**
- `Block::full_bytes()` / `Block::canonical_bytes()` do not serialize `fees_total_atoms` or `fees_miner_atoms`.
 - File: `block.rs:269-297`
3. **Full transaction canonical decoder is permissive.**
- Missing `tx-PoW` tail defaults to zero instead of rejecting.
 - File: `transaction.rs:750-760`
4. **Manual decoders allocate before bounding counts.**
- `transaction.rs:719-746`
 - `transaction.rs:246-250,` `273-292`
 - `block.rs:419-425`
5. **Malformed witness bytes are under-committed.**
- `witness_payload()` returns `None` on malformed bytes.
 - `witness_commitment_hash()` then behaves as if there were no parsed witness.
 - File: `transaction.rs:655-660, 616-623`

Serialization and Hashing Grade: 4/10

Why:

- Header, txid, and block hashing code is deterministic.
- But determinism is not enough when the committed object does not include all the bytes that validity depends on.

10. Mempool vs Block Consensus Alignment

Findings

Check	Status	Notes
Mempool rejects invalid consensus txs	Good	Calls <code>validate_transaction_with_context_for_mempool()</code>
Mempool extra policy on top of consensus	Good	Dust / standard inputs / fee floor layered on
Block validation does not trust mempool prevalidation	Good	Revalidates directly against overlay UTXO set
Reorg reinsertion	Good	Disconnected txs are re-admitted if still valid
Miner candidate assembly uses mempool state	Good	Conflict filtering and size accounting in <code>mining.rs</code>
Miner template self-validation	Missing	Candidate not run back through block validator
Policy-vs-consensus clarity	Poor comments	Dust/fee floor are described as policy-only in places even though <code>prepare_transaction_validation()</code> enforces them during block validation

Mempool / Block Alignment Grade: 7/10

Why:

- The actual code paths are mostly aligned.
- The main downgrades are missing miner self-validation and potentially misleading

comments about which rules are policy versus consensus.

11. Miner Consensus Audit

Findings

Check	Status	Notes
Uses same subsidy schedule	Good	mining.rs:45-49
Uses same size/weight/vsize limits	Good	mining.rs:108-125
Avoids double spends in template	Good	mining.rs:84-105
Uses same fee calculation model	Good enough	Pulls fee_atoms from validated entries
Self-validates template	completed Missing	Should call block validator before handing block to miners/RPC
Reward destination ownership	Critical failure	Deterministic public seed-derived reward key

Miner Consensus Grade: 3/10

Why:

- The template assembly mechanics are decent.
- The reward address design is not safe for any real network.

12. Difficulty and Proof-of-Work Audit

Findings

Check	Status	Notes
Target calculation	Good	<code>pow.rs:242-305</code>
Chainwork comparison	Good	<code>pow.rs:350-373</code>
Target bounds	Good	<code>pow.rs:391-412</code>
PoW comparison endianness	Good	<code>hash <= target</code> on same big-endian byte ordering
MTP lower bound	Good	<code>pow.rs:375-389</code> , <code>validation.rs:1220-1224</code>
Future drift ceiling	Missing	No upper bound
Testnet special rule	Good	<code>pow.rs:295-303</code> , tests exist
Validation independent of miner	Good	<code>validation.rs</code> uses <code>pow::meets_target</code> directly

PoW and Difficulty Grade: 6/10

Why:

- Retargeting and chainwork code is well-scoped and tested.
- The missing future timestamp ceiling keeps this from being mainnet-grade.

13. Chain Selection and Reorg Audit

Findings

Check	Status	Notes
Best chain by cumulative	Good	<code>pow.rs:350-373</code> , <code>chainstate.rs:327-332</code>

Check	Status	Notes
work		
Height-only bug resistance	Good	Work compare first, height second
Max reorg depth	Good	<code>chainstate.rs:323-325, 1458-1485</code>
Incremental rollback safety	Good	<code>switch_branch_incrementally,</code> <code>rollback_incremental_branch_switch</code>
Full rewrite fallback	Good	<code>replace_with_validated_branch</code>
Mempool restoration after reorg	Good	<code>node.rs:665-683</code>
Crash during reorg	Good coverage	Multiple restart/reorg tests in <code>chainstate.rs</code>
Dependency on persisted UTXO correctness	Weak	Reorg logic assumes loaded UTXOs are canonical

Chain Selection and Reorg Grade: 7/10

Why:

- The reorg code is materially better than typical early-stage chain code.
- The grade is capped by storage-trust issues on restart.

14. Network Constants and Environment Separation

Findings

Check	Status	Notes
Unique consensus ids	Good	<code>network.rs:38-57</code>

Check	Status	Notes
Unique magic bytes	Good	<code>network.rs:104-122</code>
Unique ports	Good	<code>network.rs:84-102</code>
Unique visible prefixes	Good	<code>network.rs:124-141</code>
Genesis separation	Good	<code>genesis.rs</code> per-network constants
DB path separation	Good	<code>path.rs:11-48</code>
Runtime override of mainnet consensus constants	No evidence found	Sync knobs are runtime-configurable; consensus constants are not
Prunetest-only max reorg env override	Acceptable	Non-mainnet only

Network Constants Grade: 8/10

Why:

- Network separation is one of the cleaner parts of the codebase.

15. Storage and Database Consensus Safety

Findings

Check	Status	Notes
Atomic commits	Good	<code>db.rs:694-769, 774-831</code>
Crash journal	Good	<code>CommitJournalGuard</code> usage
Block archive + LMDB	Good	during normal Both updated in one logical path

Check	Status	Notes
alignment	commit	
Schema versioning	Good	db.rs:911-935
Genesis / network metadata checks	Good	db.rs:952-1004
Startup state verification	Insufficient	Tip/header checked, UTXO contents not checked
Corrupted UTXO false-validity risk	High	See Findings B and apply_utxo_delta
Raw block archive self-sufficiency	Weak	Canonical raw block bytes omit fee metadata

Storage Consensus Safety Grade: 4/10

Why:

- Normal commit atomicity is decent.
- Restart trust and persistence integrity are not strong enough for production consensus safety.

16. Legacy Code and Bypass Audit

Findings

Search Area	Result	Risk
Legacy lock formats	Explicitly rejected (validation.rs:463-468)	Low
Legacy TSV snapshot/runtime	Quarantined /	rejected Low

Search Area	Result	Risk
layouts	(chainstate.rs:1414-1419, 1488+)	
Accept-on-error paths in validation	None found in consensus validator	Low
Internal skip-PoW helper	Exists but clearly labeled internal (validation.rs:1010-1019)	Medium if misused later
Decoder fallback accepting old shape	Transaction::from_full_bytes() missing tx-PoW tail default	High
Malformed witness collapsing to no witness in commitments	Present	High

Legacy and Bypass Risk Grade: 7/10

Why:

- The codebase is no longer carrying many obvious “legacy accept both formats” landmines.
- The biggest remaining bypass flavor is decoder permissiveness and malformed-witness under-commitment, not classic legacy-compatibility sprawl.

17. Required Test Suite

Block Tests

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
valid_bloc	Baseline	Build solved	Accepted	crates/atho-	Yes	Pass
k_accepted	block acceptance	canonical block	and state at next mutates	storage/src/chainstate.rs	(chainstate _tracks_tip	

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
		height			_and_height)	
bad_previous_hash_rejected	Parent binding	Use wrong previous_block_hash	Reject with parent mismatch	crates/athos-node/src/node.rs validation.rs	Yes (node_reject or ts_wrong_parent_hash)	Pass
bad_height_rejected	Height binding	Header height != expected height	Reject	validation.rs	Partial	Pass (partial)
bad_merkle_root_rejected	Body/header commitment	Mutate header merkle root	Reject with merkle mismatch	validation.rs	No	Missing
bad_pow_rejected	PoW enforcement	Unsolved block wrong nonce	Reject or with PoW invalid	validation.rs	Partial	Pass (partial)
bad_timestamp_rejected_future	Future cap	Timestamp drift beyond allowed ceiling	Reject	validation.rs	No	Missing
oversized_block_rejected	Size caps	Exceed raw/vsize/weight	Reject	validation.rs	Yes (oversized_)	Pass

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
cted		ght			block_*)	
block_with _no_coinbase_rejected	Coinbase presence	First tx non-coinbase	Reject	validation.rs	Partial	Missing explicit
block_with _two_coinbases_rejected	Unique coinbase	First and later tx are coinbase	Reject	validation.rs	No	Missing
coinbase_not_first_rejected	Coinbase position	Put coinbase at index > 0	Reject	validation.rs	No	Missing
duplicate_txid_rejected	Duplicate detection	tx Same twice	tx Reject	validation.rs	No	Missing
duplicate_input_rejected_in_block	Intra-block double spend	Two txs spend same outputpoint	Reject	validation.rs	No	Missing
overclaim_reward_rejected	Inflation prevention	Coinbase output > subsidy	Reject	validation.rs	Partial	Pass (partial)

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
		fees				
extra_money_created_rejected	Fee exactness	Outputs exceed inputs + subsidy	Reject	validation.rs	Partial	Pass (partial)
missing_utxo_in_block_rejected	UTXO existence	Spend non-existent output	Reject	validation.rs	Partial	Pass (partial)
spent_utxo_in_block_rejected	Double-spend across blocks	Spend already-spent output	Reject	validation.rs / chainstate.rs	Partial	Pass (partial)
immature_coinbase_spending_rejected	Maturity rule	Spend immature coinbase	Reject	validation.rs	No	Missing
invalid_signature_in_block_rejected	Signature enforcement	Corrupt Falcon signature	Reject	validation.rs	Partial	Pass (partial)

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
coinbase_witness_nonempty_witness_rejected	Coinbase strictness	Add witness bytes to coinbase	Reject	validation.rs	No	Missing
coinbase_witness_nonzero_txpow_rejected	Coinbase strictness	Set tx_pow_nonce/bits on coinbase	Reject	validation.rs	No	Missing

Transaction Tests

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
valid_transaction_accepted	Baseline validity	Canonical tx spend with correct witness and fee	Accept	validation.rs / mempool.rs	Yes	Pass
invalid_signature_rejected	Falcon validity	Corrupt signature bytes	Reject	validation.rs	Partial	Pass (partial)
wrong_public_key_rejected	Ownership binding	Use mismatched Falcon pubkey	Reject	validation.rs	Yes (wrong_public_key_for_standard_output_is_rejected)	Pass

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
wrong_address_rejected	Lock mismatch	Unlocking script from lock	differs from UTXO Reject	validation.rs	Partial	Pass (partial)
wrong_network_prefix_rejected	Cross-network replay	Mainnet-signed tx on testnet	Reject	validation.rs	Yes	Pass
missing_input_rejected	UTXO existence	Spend outpoint	missing Reject	validation.rs	Partial	Pass (partial)
duplicate_input_rejected	Intra-tx double spend	Reuse outpoint	same twice Reject	validation.rs	No direct test	Missing
negative_output_rejected	Impossible with u64	N/A	in Rust type system N/A	validation.rs	N/A	Type-safe
zero_output_rejected	Zero-value output rule	Output = 0	value = Reject	validation.rs	No direct test	Missing
dust_output_rejected	Dust rule	Output floor	below Reject	validation.rs / mempool.rs	Yes	Pass
fee_below_floor_rejected	Fee floor	Fee required minimum	below Reject	validation.rs	Partial	Pass (partial)
output_sum_gt_input_sum_rejected	Inflation	Overspend	Reject	validation.rs	Partial	Pass

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
cted	prevention	outputs				(partial)
oversized_transaction_rejected	Size cap	Exceed raw/vsize cap	tx Reject	validation.rs	Yes	Pass
malformed_full_serialization_rejected	Parser strictness	Corrupt canonical bytes	tx Reject	transaction.rs / service.rs	Partial	Pass (partial)
missing_txpow_tail_rejected	Full decoder strictness	Remove PoW tail full tx bytes	tx- from Reject	transaction.rs	No	Missing
extra_unknown_consensus_bytes_rejected	Canonical parse	Append after tx	bytes Reject	transaction.rs / service.rs	Partial	Pass (partial)
missing_required_fields_rejected	Parser hardening	Truncate witness outputs	or Reject	transaction.rs	Yes (witness_payload_rejects_truncated_payload)	Pass
replay_from_another_network_rejected	Replay protection	Re-sign for wrong network	for Reject	validation.rs	Yes	Pass

Monetary Tests

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
reward_at_height_0_correct	Genesis subsidy	Query subsidy 0	at Exact value	subsidy.rs	Yes	Pass
reward_at_normal_height_correct	Standard subsidy	era Query mid-era height	Exact value	subsidy.rs	Yes	Pass
reward_at_halving_boundary_correct	Boundary behavior	Query first block after halving	Exact value	subsidy.rs	Yes	Pass
reward_after_halving_correct	Post-boundary behavior	Query later height	Exact value	subsidy.rs	Yes	Pass
tail_emission_correct	Tail era	Query deep height	Exact amount	tail subsidy.rs	Yes	Pass
max_supply_cannot_be_exceeded	Cap enforcement	N/A because no cap	Not applicable hard documented no-cap	/subsidy.rs, params.rs	N/A	Design-specific
coinbase_cannot_claim_more_than_subsidy	Inflation prevention	Overclaim block	Reject	validation.rs	Partial	Pass (partial)

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
ids_plus_fees						
fee_calculation_exact	No double-count/underflow	Build exact-fee tx/block	Exact fee result	validation.rs	Partial	Pass (partial)
atomic_precision_exact	Integer-only accounting	Sum/format atoms	No rounding drift in consensus	constants.rs, subsidy.rs	Yes	Pass
no_rounding_inflation	No float use	Checked integer arithmetic	No inflation	validation.rs, subsidy.rs	Yes	Pass

UTXO Tests

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
utxo_created_after_block_acceptance	Output creation	Connect valid block	New outputs present	utxo.rs / chainstate.rs	Yes	Pass
utxo_spent_after_validation_transaction	Spend path	Connect spending block	Spent output removed	utxo.rs	Yes	Pass

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
utxo_not_spent_after_failed_transaction	Rollback safety	Fail during apply	Original UTXO preserved	utxo.rs / chainstate.rs	Yes	Pass
utxo_not_muted_after_failed_block	State cleanliness	Invalid block	Chainstate unchanged	chainstate.rs	Yes (invalid_block_is_rejected_without_muting_chainstate)	Pass
double_spend_same_block_rejected	Intra-block conflict	Two spend outpoint	Reject same	validation.rs	No direct test	Missing
double_spend_across_blocks_rejected	Cross-block conflict	Spend already-spent UTXO	Reject	chainstate.rs	Partial	Pass (partial)
coinbase_maturity_enforced	Coinbase lockup	Spend immature coinbase	Reject	validation.rs, utxo.rs	Partial	Pass (partial)
reorg_restores_utxos_correctly	Reorg accounting	Switch to better fork	Old spends undone, new	chainstate.rs	Yes	Pass

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
			spends applied			
crash_does_not_corrupt_chainstate	Restart safety	Fault inject commit / dirty restart	Recover or fail closed	chainstate.rs, db.rs	Partial	Pass (partial)
corrupted_persisted_undo_detected_on_restart	Startup integrity	Tamper LMDB UTXO value	Detection and rebuild/fail	db.rs, chainstate.rs	No	Missing

Serialization Tests

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
same_tx_same_txid	Canonical txid	Encode tx twice	same Same txid	transaction.rs	Yes	Pass
same_block_same_hash	Canonical hash	block Encode header twice	Same block hash same	block.rs	Yes	Pass
field_order_cannot_change_hash	Hash stability	Reorder encoding fields adversarial test	Different in / invalid	transaction.rs, block.rs, adversarial tests	Partial	Pass (partial)
missing_fields	Parse strictness	Truncate	Reject	transaction.rs,	Partial	Pass

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
elds_rejected		tx/block bytes		block.rs		(partial)
extra_fields_handled_safely	Parse strictness	Append bytes	Reject	service.rs, transaction.rs, block.rs	Partial	Pass (partial)
legacy_formats_rejected	Canonical-only decode	Use legacy lock / legacy tx shape	Reject	validation.rs, service.rs	Partial	Pass (partial)
atx2_canonical_full_tx_enforced	Full bytes exactness	Non-canonical but parseable full tx	Reject	transaction.rs, service.rs	Partial	Pass (partial)
binary_codec_roundtrip_exact	Stable storage/wire bytes	Roundtrip tx/block bytes	Equal bytes	transaction.rs, block.rs	Yes	Pass
txpow_tail_required_in_full_tx	Decoder strictness	Omit tx-PoW tail	Reject	transaction.rs	No	Missing
fees_metadata_not_needed_for_block_valid	Remove under-commitment	Validate from canonical bytes only	Same validity outcome	validation.rs, block.rs	No	Missing

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
-----------	---------	-------	-----------------	---------------	-------------------	--------

ity

Falcon Tests

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
valid_falcon _signature_accepted	Baseline	Canonical signed tx	Accept	validation.rs	Yes	Pass
invalid_falcon signature_rejected	Integrity	Corrupt signature	Reject	validation.rs	Partial	Pass (partial)
truncated_signature_rejected	Parser hardening	Short signature length	Reject	transaction.rs / validation.rs	Yes (witness parser)	Pass
oversized_signature_rejected	Parser hardening	Long signature length	Reject	transaction.rs	Yes	Pass
signature_for_different_message_rejected	Digest correctness	Re-sign altered tx	Reject	validation.rs	Partial	Pass (partial)
signature_for_different_	Tx binding	Swap body	Reject	validation.rs	Partial	Pass (partial)

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
tx_rejected		under same witness				
signature_for_different_network_rejected	Replay protection	Mainnet sig testnet	on Reject	validation.rs	Yes	Pass
signature_cache_cannot_bypass_verification	Cache safety	N/A cache	No no bypass possible	N/A	N/A	N/A
full_signature_retrieval_verified	Persistence safety	Store/load tx with witness	Same bytes	transaction.rs, storage tests	Partial	Pass (partial)

Reorg Tests

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
longer_valid_chain_wins	Canonical fork choice	Better-work branch	Reorg	chainstate.rs	Yes	Pass
higher_work_wins_over_the	Chainwork priority	Shorter but more-work branch	Prefer higher work	pow.rs, sync.rs	Yes	Pass

Test Name	Purpose	Setup	Expected Result	Code Location	Currently Exists?	Status
right_only						
invalid_longer_chain_rejected	Reorg validation	Bad candidate branch	Keep current	chainstate.rs	Yes	Pass
deep_reorg_beyond_limit_rejected	Finalization / safety	Exceed max reorg depth	Reject	chainstate.rs	Yes	Pass
reorg_rolls_back_utxos_correctly	Accounting	Disconnect/reconnect branch	Exact UTXO restore	chainstate.rs	Yes	Pass
reorg_restores_mempool_transactions	Runtime correctness	Reorg disconnects mempool-worthy txs	Re-admit valid txs	node.rs	Partial	Pass (partial)
reorg_cannot_bypass_coinbase_maturity	Maturity safety	Reorg around immature spend	Reject	chainstate.rs, validation.rs	No	Missing
checkpoints_enforced	Finalization boundary	Fork finalized checkpoint	Reject	chainstate.rs	Partial	Pass (partial)

18. Fuzz and Adversarial Testing

Fuzzing Plan

Target	Input Mutations	Must Never Happen	Recommended Harness
Transaction decoder (Transaction::from_full_bytes)	Truncation, count inflation, bad lengths, bad tx-PoW tail	Panic, OOM, accept malformed tx	cargo-fuzz target in atho-core
Witness decoder (TxWitness::from_bytes)	Signature length, pubkey length, ref counts, additional signer counts	Panic, parse malformed witness as valid	cargo-fuzz
Block decoder (Block::from_canonical_bytes)	Huge tx counts, truncated tx bytes, nested malformed txs	Panic, OOM, accept malformed block	cargo-fuzz
Compact-size encoder/decoder surfaces	Boundary overflow-style values	ints, Divergent between nodes	parse Property tests
Address parser	Wrong prefix, wrong checksum, huge strings	Panic or network accept	cross-cargo-fuzz + proptest
Tx-PoW preimage / parser	Random witness/txpow combinations	Accept wrong nonce/bits	Property tests
UTXO key/value parser	Corrupted LMDB bytes	Panic or silent false validity	Storage-level fuzz harness
P2P block payload decode	Malformed bincode payloads, huge vectors	Panic, memory blowup, wrong	atho-p2p fuzz

Target	Input Mutations	Must Never Happen	Recommended Harness
		network accept	harness
Compact reconstruction	Same txid / different block witness or tx-PoW variants	Reconstruct wrong block variant silently	Adversarial unit tests
Snapshot deserialize	bundle Corrupt bundles, wrong network, wrong tip	Crash, load false-valid chainstate	atho-node fuzz + adversarial tests

Adversarial Cases To Add Immediately

- Same header, same txids, different tx_pow_nonce / tx_pow_bits
- Same header, same txids, different fees_total_atoms / fees_miner_atoms
- Non-empty malformed coinbase witness bytes
- Truncated full tx bytes that omit tx-PoW fields
- Very large input_count, output_count, tx_count, additional_group_count
- Corrupted persisted LMDB UTXO entry surviving restart

19. Grade Every Subsystem

Subsystem	Grade	Mainnet Ready?	Biggest Risk	Required Fix
Block validation	5/10	No	Validity depends on Commit tx-PoW / stop uncommitted fields	trusting fee metadata
Transaction validation	7/10	Not alone	Decoder strictness and tx-PoW under-commitment	Strict full decode + commitment fix
UTXO	5/10	No	Dirty restart trusts persisted	Persist UTXO root or

Subsystem	Grade	Mainnet Ready?	Biggest Risk	Required Fix
accounting			UTXOs too much	rebuild/verify on startup
Monetary policy	8/10 Mostly		No cap by design; fee metadata coupling	Keep issuance math, decouple fees from block metadata
Coinbase validation	4/10 No		Missing coinbase witness/tx-PoW strictness	Enforce empty witness and zero tx-PoW
Falcon signature validation	8/10 Mostly		Depends on surrounding serialization correctness	Keep, add more malformed-block tests
Serialization/ hashing	4/10 No		Block header under-commits consensus data	Redesign witness commitment / strict decode
Mempool alignment	7/10 Almost		Comments and miner self-validation gap	Self-validate block templates
Miner consensus logic	3/10 No		Deterministic public reward keys	Use configured private reward destination
PoW/difficulty	6/10 Not yet		No future timestamp ceiling	Add deterministic future cap
Chain selection/reorgs	7/10 Close		Restart depends on persisted UTXO trust	Verify/rebuild UTXOs on startup
Network constants	8/10 Yes		Few issues here	Keep
Storage/LMDB	4/10 No		UTXO integrity not re-proven	UTXO root + fail-closed

Subsystem	Grade	Mainnet Ready?	Biggest Risk	Required Fix
safety			on dirty restart	recovery
Legacy bypass risk	7/10	Mostly	Permissive full-tx decode / malformed witness commitment collapse	Remove fallback paths
API/RPC consensus safety	7/10	Mostly	External callers can feed consensus objects; no admin bypass found	Keep canonical checks, add template self-validation
Test coverage	7/10	Not yet	Missing tests for the actual blockers above	Add tests from Section 17

20. Mainnet Launch Decision

Decision: MAINNET BLOCKED

What must be fixed before mainnet

1. Commit tx-PoW bytes into the block commitment path.
2. Remove deterministic/public mining reward keys.
3. Verify or rebuild persisted UTXO state on dirty startup.
4. Add strict coinbase witness / coinbase tx-PoW rules.
5. Make full transaction decoding strict.
6. Bound manual decode allocations.
7. Add future timestamp ceiling.
8. Add the missing tests in Section 17.

What should be fixed immediately after mainnet if launch were ever forced

- Compact-block ambiguity hardening
- Stronger fuzzing and malformed-input CI

- Better differentiation between relay policy and consensus comments
- Full storage integrity metrics / offline verifier

What tests must pass before mainnet

- All missing blocker tests listed in Section 17
- Adversarial variants proving there is no alternate-valid raw payload for one header hash
- Dirty restart / corrupted UTXO recovery tests
- Candidate miner template self-validation tests

Areas that need a second audit

- P2P compact block reconstruction after tx-PoW commitment changes
- Snapshot bootstrap trust model
- Release engineering / reproducible builds
- GPU miner compatibility after tx-PoW commitment changes

Is an external audit recommended?

Yes. The under-commitment issue around tx-PoW and block metadata is exactly the kind of thing that deserves an external second set of eyes before any irreversible launch.

21. Required Fix Patches

Fix 1: Commit all consensus-relevant transaction bytes into the block commitment path

- File: `crates/atho-core/src/transaction.rs`
- Function: `Transaction::witness_commitment_hash`
- Bug:
 - The function commits base bytes and parsed witness commitment bytes only.
 - It omits `tx_pow_nonce` and `tx_pow_bits`.
 - Malformed witness bytes can collapse to the same commitment as “no parsed witness”.
- Why consensus-critical:
 - Block validity depends on tx-PoW.
 - Two payloads with the same header can differ in tx-PoW validity.

- Safe patch:

```
pub fn witness_commitment_hash(&self) -> [u8; 48] {
    let mut hasher = Sha3_384::new();
    self.update_base_hasher(&mut hasher);
    hasher.update((self.witness.len() as u32).to_le_bytes());
    hasher.update(&self.witness);
    hasher.update(self.tx_pow_nonce.to_le_bytes());
    hasher.update([self.tx_pow_bits]);
    hasher.finalize().into()
}
```

- Why low-regression:
- This preserves deterministic hashing while finally binding the same bytes validation depends on.
- Pre-mainnet, changing block commitments is the right time to do it.
- Test to add:
- `block_header_commitment_changes_when_txpow_changes`
- `malformed_nonempty_witness_changes_witness_root`

Fix 2: Remove fee metadata from consensus validity

- Files:
- `crates/atho-storage/src/validation.rs`
- `crates/atho-node/src/mining.rs`
- `crates/atho-p2p/src/protocol.rs`
- Functions:
- `validate_block_with_context_and_schedule`
- `collect_prepared_block_transactions_with_schedule`
- Bug:
- `fees_total_atoms / fees_miner_atoms` affect validity but are not committed by the header and are not in canonical block bytes.

- Why consensus-critical:
- Same header + same transactions can produce different validity outcomes depending on out-of-band fee metadata.
- Safe patch:
- Compute fees from transactions inside block validation.
- Compare coinbase output amount directly to `subsidy + computed_sum_fees`.
- Stop using `block.fees_*` as validity inputs; keep them as post-validation cached metadata only.

Pseudo-change:

```
let computed_fees = sum_fees;
let expected_reward = subsidy::block_subsidy_atoms_for_network(network, height)
    .checked_add(computed_fees)
    .ok_or(ValidationError::CoinbaseRewardMismatch)?;
validate_coinbase_transaction_strict(&block.transactions[0], expected_reward,
height, schedule)?;
```

- Why low-regression:
- It removes ambiguous out-of-band consensus input rather than adding new consensus state.
- Test to add:
- `block_fee_metadata_cannot_change_validity`

Fix 3: Replace deterministic public reward keys

- File: `crates/atho-node/src/mining.rs`
- Function: `reward_target_for_height`
- Bug:
- Reward keypair is derived from public `(network, height)` data.
- Why consensus-critical / launch-critical:
- Mined rewards are economically stealable by anyone.

- Safe patch:
- Remove `reward_target_for_height`.
- Require a configured reward address / payment digest from node config or wallet.
- On mainnet, refuse template construction without an explicit reward destination.

Pseudo-change:

```
let reward_script = node
    .configured_mining_reward_script()
    .ok_or(NodeError::Configuration("missing mining reward address"))?;
```

- Why low-regression:
- Changes miner output destination only; does not weaken validation.
- Test to add:
- `mainnet_candidate_block_requires_configured_reward_destination`

Fix 4: Verify or rebuild UTXO state on dirty startup

- Files:
- `crates/atho-storage/src/db.rs`
- `crates/atho-storage/src/chainstate.rs`
- Functions:
- `run_startup_consistency_checks`
- `verify_persisted_chainstate_consistency`
- `load_persisted_chainstate`
- Bug:
- Startup check proves tip/header consistency, not UTXO consistency.
- Why consensus-critical:
- Corrupted UTXOs can make a node accept or mine invalid spends after restart.
- Safe patch:

- Persist a deterministic UTXO root/hash in `ChainstateSnapshot` or metadata.
- On dirty startup, either:
 1. recompute and compare the UTXO root, or
 2. rebuild the UTXO set from canonical blocks and atomically replace chainstate.
- Treat missing spent-key deletes as corruption outside explicit recovery/dev mode.
- Why low-regression:
 - Uses canonical blocks as source of truth.
 - Affects startup/recovery, not steady-state consensus math.
 - Test to add:
 - `dirty_restart_detects_corrupted_utxo_and_repairs_or_fails_cl`
`osed`

Fix 5: Make coinbase witness/tx-PoW strict

- File: `crates/atho-storage/src/validation.rs`
- Function: `validate_coinbase_transaction_with_schedule`
- Bug:
 - Coinbase does not require empty witness or zero tx-PoW fields.
- Why consensus-critical:
 - Coinbase serialization and witness commitment semantics are under-specified.
- Safe patch:

```
if !tx.witness.is_empty() || tx.tx_pow_nonce != 0 || tx.tx_pow_bits != 0 {
    return Err(ValidationError::InvalidCoinbase);
}
```

- Why low-regression:
 - Coinbase transactions do not need witness or tx-PoW under current design.
 - Test to add:
 - `coinbase_with_nonempty_witness_rejected`

- `coinbase_with_nonzero_txpow_rejected`

Fix 6: Make canonical full transaction decoding strict

- File: `crates/atho-core/src/transaction.rs`
- Function: `Transaction::from_full_bytes`
- Bug:
- Missing tx-PoW fields default to `(0, 0)`.
- Why consensus-critical:
- Non-canonical payloads can be parsed as canonical objects.
- Safe patch:

```
let lock_time = read_u32(bytes, &mut offset)?;
let tx_pow_nonce = read_u64(bytes, &mut offset)?;
let tx_pow_bits = *bytes.get(offset)?;
offset += 1;
if offset != bytes.len() {
    return None;
}
```

- Why low-regression:
- Aligns decoder with encoder exactly.
- Test to add:
- `full_tx_missing_txpow_tail_is_rejected`

Fix 7: Bound manual decoder allocations before allocation

- Files:
- `crates/atho-core/src/transaction.rs`
- `crates/atho-core/src/block.rs`
- Functions:
- `TxWitness::from_bytes`
- `Transaction::from_full_bytes`

- `Block::from_canonical_bytes`
- Bug:
- Counts are trusted before allocation.
- Why consensus-critical:
- Malformed inputs should never be able to exhaust memory or crash the node.
- Safe patch:
- Before any `Vec::with_capacity(count)`, derive a maximum feasible count from remaining bytes and fixed minimum element sizes.
- Reject payloads whose counts exceed what the payload length could possibly encode.
- Why low-regression:
- Tightens parser rejection only for impossible or abusive inputs.
- Test to add:
- `block_decoder_rejects_impossible_tx_count_before_allocation`
- `tx_decoder_rejects_impossible_input_count_before_allocation`

Fix 8: Add a future timestamp ceiling

- Files:
- `crates/atho-core/src/constants.rs`
- `crates/atho-storage/src/validation.rs`
- Functions:
- `validate_contextual_header_precheck`
- Bug:
- No upper future timestamp bound exists.
- Why consensus-critical:
- Time manipulation can destabilize chain progression and difficulty.
- Safe patch:
- Add a deterministic ceiling such as:

```

const MAX_FUTURE_BLOCK_DRIFT_SECONDS: u64 = 7_200;
if let Some(mtp) = pow::median_time_past_from_blocks(previous_blocks) {
    if block.header.timestamp >
mtp.saturating_add(MAX_FUTURE_BLOCK_DRIFT_SECONDS) {
        return Err(ValidationError::InvalidBlockTimestamp);
    }
}

```

- Why low-regression:
- Keeps the rule deterministic across nodes.
- Test to add:
- `future_dated_block_above_drift_limit_rejected`

22. Final Deliverables

Critical Fixes

1. Commit tx-PoW bytes in the witness commitment path.
2. Remove fee metadata from consensus validity or commit it canonically.
3. Replace deterministic public mining reward keys.
4. Verify or rebuild UTXO state on dirty startup.
5. Enforce empty coinbase witness and zero coinbase tx-PoW.

High-Priority Fixes

1. Make `Transaction::from_full_bytes()` strict.
2. Bound decoder allocations before allocation.
3. Add future timestamp ceiling.
4. Self-validate miner block templates before exposing them.
5. Harden compact-block reconstruction against txid-only ambiguity.

Missing Tests

- All tests marked `Missing` in Section 17, especially:
- tx-PoW commitment / block identity tests

- fee metadata validity independence
- coinbase strictness tests
- dirty-restart corrupted UTXO detection
- strict decoder tests
- future timestamp ceiling tests

Production-Readiness Grade

- **4/10**

Mainnet Launch Recommendation

- **MAINNET BLOCKED**

Subsystem Grade Table

- See Section 19.

Regression Test Plan

- See Section 17.

Fuzz Test Plan

- See Section 18.

Summary of All Consensus Risks

The dominant pattern in the current Atho codebase is not “missing basic validation.” It is **under-committed consensus data** and **restart-state trust**:

1. Block validity depends on bytes the header does not commit.
2. Miner rewards are not actually private to the miner.
3. Restarted nodes do not fully re-prove UTXO correctness before trusting persisted state.

Until those are fixed, Atho should not launch a value-bearing mainnet.

Appendix E: Open Questions

1. Which snapshot trust model should Atho standardize for release-quality bootstrap: operator-pinned signer keys, built-in release keys, or both?
2. How much miner-vs-validator differential testing should be mandatory before mainnet?
3. Should the project publish an external protocol specification that freezes consensus bytes independent of the Rust implementation?
4. How much compact-block complexity is worth retaining versus simpler but more bandwidth-heavy relay?
5. Will future privacy or instant-accept layers remain clearly non-consensus overlays, or will they try to modify base validation?
6. Is Prunetest sufficient for long-running storage stress, or is a more explicit fault-injection network mode warranted?

Appendix F: Future Improvements

- Run short fuzz smoke jobs in pull-request CI and longer fuzz sessions on scheduled hardening lanes.
- Expand wallet/node and miner/validator differential testing.
- Add broader reorg disconnect/reconnect failpoints and assert exact recovery.
- Tighten production startup so all convenience constructors become fail-closed `try_*` paths.
- Publish a standalone Atho consensus specification derived from the code and locked by test vectors.
- Strengthen snapshot bootstrap from simple hash pinning toward a signed-distribution model.
- Commission at least one external consensus and cryptography review before public mainnet launch.
- Continue reducing clone-heavy or stale-cache read paths that can mask correctness issues

under load.

- Expand operational docs for bootstrap nodes, explorers, wallet endpoints, and release signing.